

Systemes Embarqués

Introduction aux Systemes Embarqués



Licence en Électronique et Systemes Embarqués



Année Universitaire 2025-2026



Pr. Khalid SALMI



Parties non incluses dans ce document (faites dans le cours) :

1. Introduction

2. Rappel Electronique Numérique (logique combinatoire et séquentielle)

3. Les registres

Les mémoires

Caractéristiques d'une mémoire

- **La capacité** : c'est le nombre total de bits que contient la mémoire. Elle s'exprime aussi souvent en octet.
- **Le format des données** : c'est le nombre de bits que l'on peut mémoriser par case mémoire. On dit aussi que c'est la largeur du mot mémorisable.
- **Le temps d'accès** : c'est le temps qui s'écoule entre l'instant où a été lancée une opération de lecture en mémoire et l'instant où l'information est disponible sur le bus de données.
- **Le temps de cycle** : il représente l'intervalle minimum qui doit séparer deux demandes successives de lecture ou d'écriture.
- **Le débit** : c'est le nombre maximum d'informations lues ou écrites par seconde.
- **Volatilité** : elle caractérise la permanence des informations dans la mémoire. Une mémoire est volatile si son contenu est effacé lors de la mise hors tension (**les RAM**), Mémoire est non volatile si son contenu est maintenu lors de la mise hors tension (**les ROM**)

Les types de mémoires

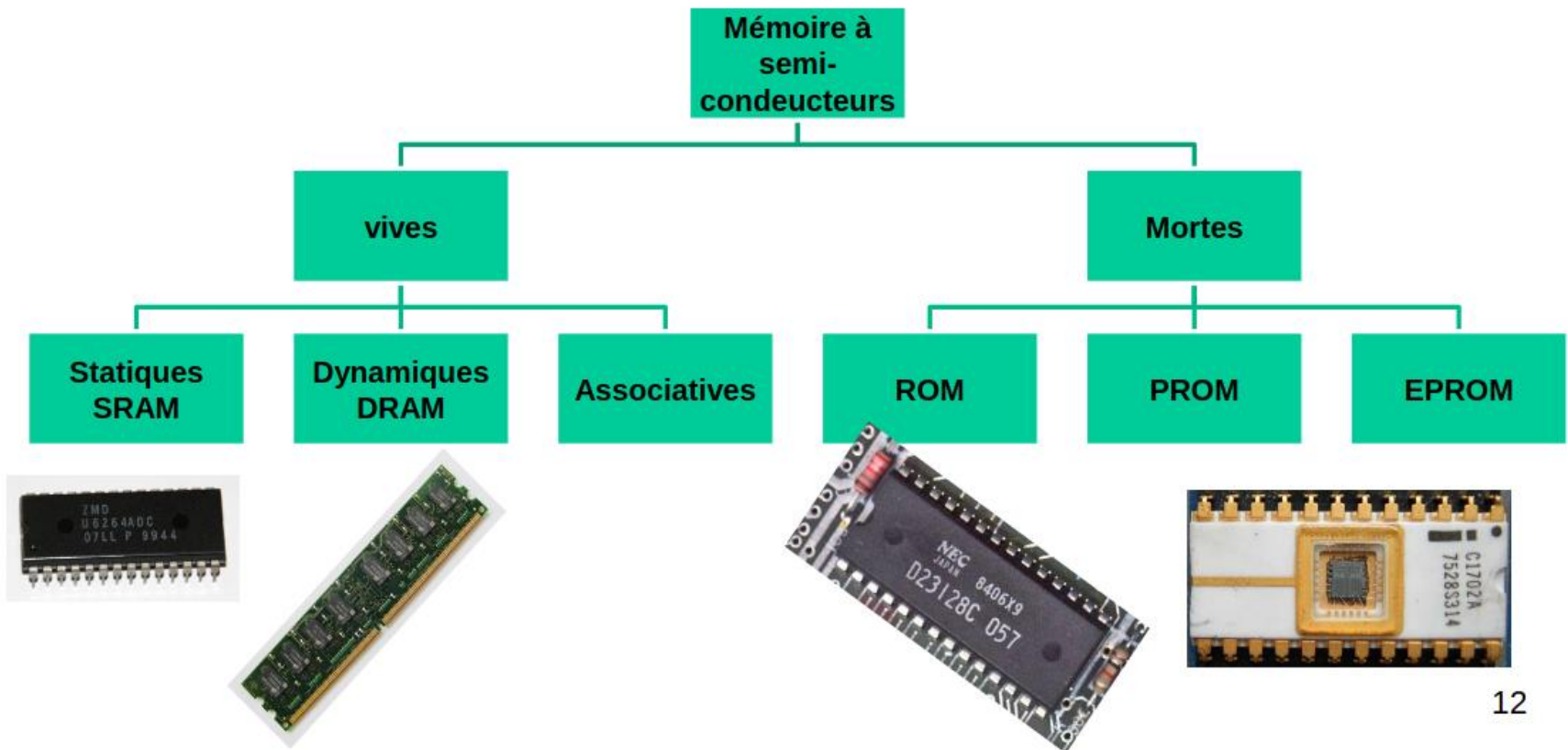
- Les registres.
- La mémoire cache.
- La mémoire principale.
- La mémoire d'appui.
- La mémoire de masse.



Classement des mémoires

Les mémoires peuvent être classés en trois catégories selon la technologie utilisée :

- Mémoire à semi-conducteur (mémoire centrale, ROM, PROM,.....) : très rapide mais de taille réduite.
- Mémoire magnétique (disque dur, disquette,...) : moins rapide mais stocke un volume d'informations très grand.
- Mémoire optique (DVD, CDROM,..)



12

La mémoire centrale RAM : Random Acces memory (Mémoire à accès aléatoire)

La mémoire centrale (MC) représente l'espace de travail de l'ordinateur (calculateur). C'est l'organe principal de rangement des informations utilisées par le processeur.

Dans une machine (ordinateur / calculateur) pour exécuter un programme il faut le charger (copier) dans la mémoire centrale. La mémoire centrale est volatile : la conservation de son contenu nécessite la permanence de son alimentation électrique.

Il existe deux grandes familles des mémoires centrales : les mémoires statiques (**SRAM**) et les mémoires dynamiques (**DRAM**).

- **Les mémoires statiques** sont à base de bascules de type D , elles possèdent un faible taux d'intégration mais un temps d'accès rapide (Utilisation pour les mémoires cache).
- **Les mémoires dynamiques** à base de condensateurs , ces mémoires possèdent un très grand taux d'intégration, elles sont plus simples que les mémoires statiques mais avec un temps d'accès plus long .

Capacité d'une MC

On peut exprimer la capacité de la mémoire centrale soit en nombre de mots mémoire ou en bits (octets, kilo-octets,...)

La capacité = 2^k Mots mémoire

La capacité = $2^k * n$ Bits ou n = le nombre de bit de mots

Exemple :

Dans une mémoire la taille du bus d'adresses $K=14$ et la taille du bus de données $n=4$. Calculer la capacité de cette mémoire ?

$$C = 2^{14} = 16384 \text{ Mots de 4 bits}$$

$$C = 2^{14} * 4 = 65536 \text{ Bits} = 8192 \text{ Octets} = 8 \text{ Ko}$$

Extension et conception des MC

On veut réaliser une mémoire de capacité C , mais nous disposons uniquement de boîtiers (des circuits) de taille inférieure. Comment peut-on procéder ?

Décodeur d'adresses

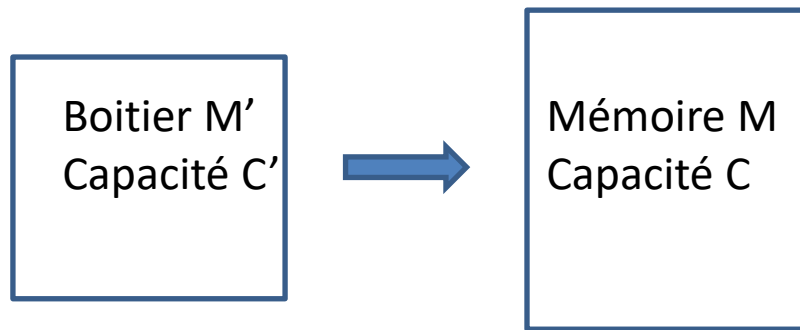
Les boîtiers mémoire possèdent une broche notée **CS : Chip Select**. Lorsque cette broche est active (souvent sur état bas), le circuit peut être lu ou écrit. Lorsqu'elle est inactive (état haut), le circuit est exclu du service : ses broches de données passent à l'état de haute impédance : tout se passe comme si la mémoire était **déconnectée** du bus de données du microprocesseur, d'où la possibilité de connecter plusieurs boîtiers mémoire sur un même bus : un seul signal CS ou CE (Chip Enable) doit être actif à un instant donné pour éviter les conflits entre les différents boîtiers.

chaque boîtier doit répondre à une plage d'adresses différente des autres boîtiers (on affecte à chaque boîtier une plage d'adresses différente des autres boîtiers)

Soit M une mémoire de capacité C , tel que m est le nombre de mot et n la taille d'un mot.

Soit M' un boîtier de capacité C' , tel que m' le nombre de mot et n' la taille d'un mot.

- On suppose que $C > C'$ ($m \geq m'$, $n \geq n'$)
- Quel est le nombre de boîtiers M' nécessaire pour réaliser la mémoire M ?



- Pour connaître le nombre de boîtiers nécessaire, il faut calculer les deux facteurs suivants :

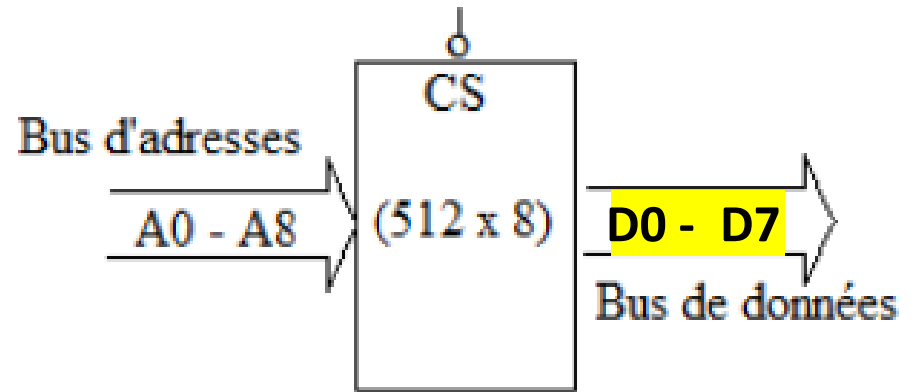
$$P = m/m'$$

$$Q = n/n'$$

- **P** : permet de déterminer le nombre de boîtiers M' nécessaire pour obtenir le nombre de mots de la mémoire M (extension lignes).
- **Q** : permet de déterminer le nombre de boîtier M' nécessaire pour obtenir la **taille de mot** de la mémoire M (extension mots ou extension colonnes).

Exemple 1:

Déterminer la Plage d'adresses de la mémoire donnée par la figure ci contres :



La plage d'adressage ,c'est le champ d'adresses occupé par un circuit (boîtier) mémoire de l'adresse minimale (adresse basse) à l'adresse maximale (adresse haute).

adresse	A8	A7	A6	A5	A4	A3	A2	A1	A0	Adresses décimales	Adresses héxa.	Plage d'adresses
minimale	0	0	0	0	0	0	0	0	0	0	\$000	0 - 511
maximale	1	1	1	1	1	1	1	1	1	511	\$1FF	\$000-\$1FF

Exemple2

On désire construire une **mémoire de 1Ko** à partir de **2 boîtiers de 512 octets** chacun. Pour les 2 circuits; on aura donc une capacité totale de 1024 (2^{10}) octets c'est à dire une plage d'adresses de 0 (adresse minimale) à 1023 (adresse maximale).

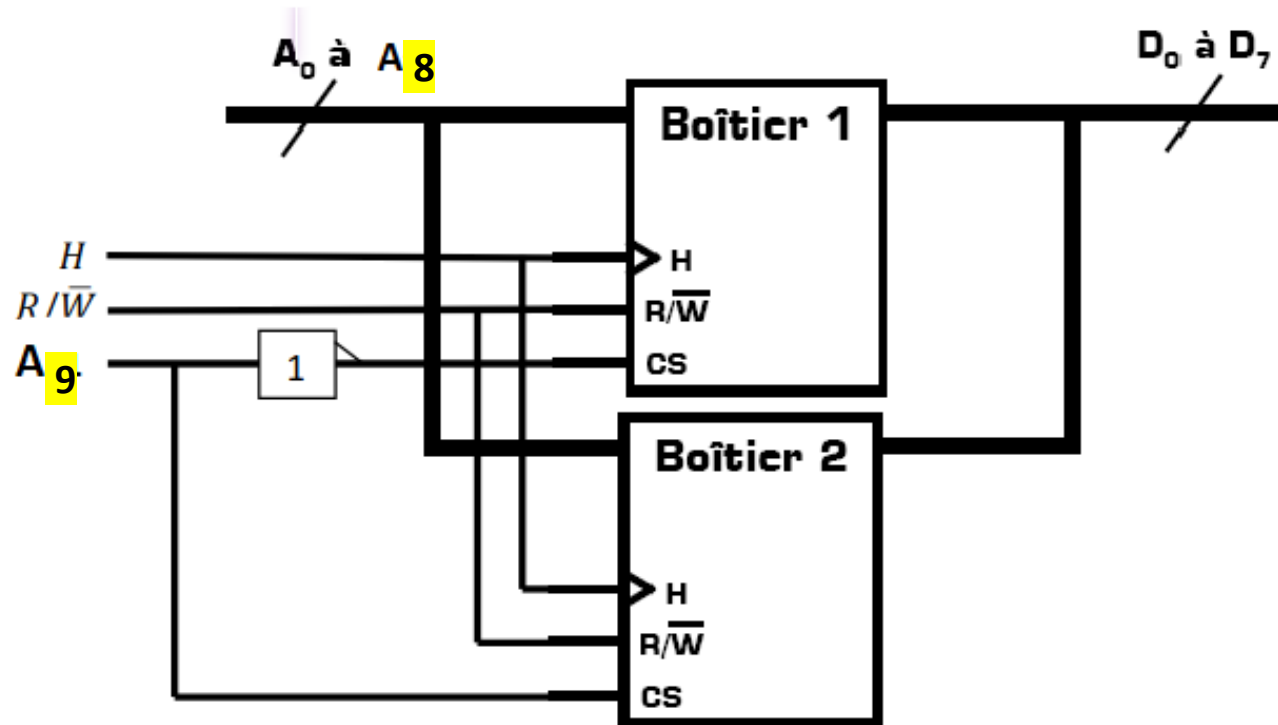
Pour adresser toute la plage d'adresses il nous faut un bus d'adresses de 10 lignes (A0 - A9). La ligne d'adresse (A9) servira à sélectionner un boîtier parmi les deux.

Correction:

On place un x pour toutes les lignes d'adresses non utilisées pour la sélection du boîtier. L'adresse minimale est obtenue en supposant que tous les x sont à un niveau 0. Pour l'adresse maximale, les x représentent des niveaux 1.

La ligne A9 détermine le boîtier sélectionné.

A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	Adresse minimale	Adresse maximale	Boîtier	Plage d'adresses
0	x	x	x	x	x	x	x	x	x	\$000 (0) ₁₀	\$1FF (511) ₁₀	A	\$000-\$1FF
1	x	x	x	x	x	x	x	x	x	\$200 (512) ₁₀	\$3FF (1023) ₁₀	B	\$200-\$3FF



Exemple 3

- Réaliser une mémoire de 1Ko (la taille d'un mot est de 8 bits) en utilisant des boîtiers de taille 256 mots de 8 bits ?

Donnée l'implémentation de cette mémoire en utilisant un décodeur

Correction:

(m,n)=(1024,8) correspond à une taille du bus d'adresses de 10 bits (A9...A0), taille du bus de données est de 8 bits (D7....D0)

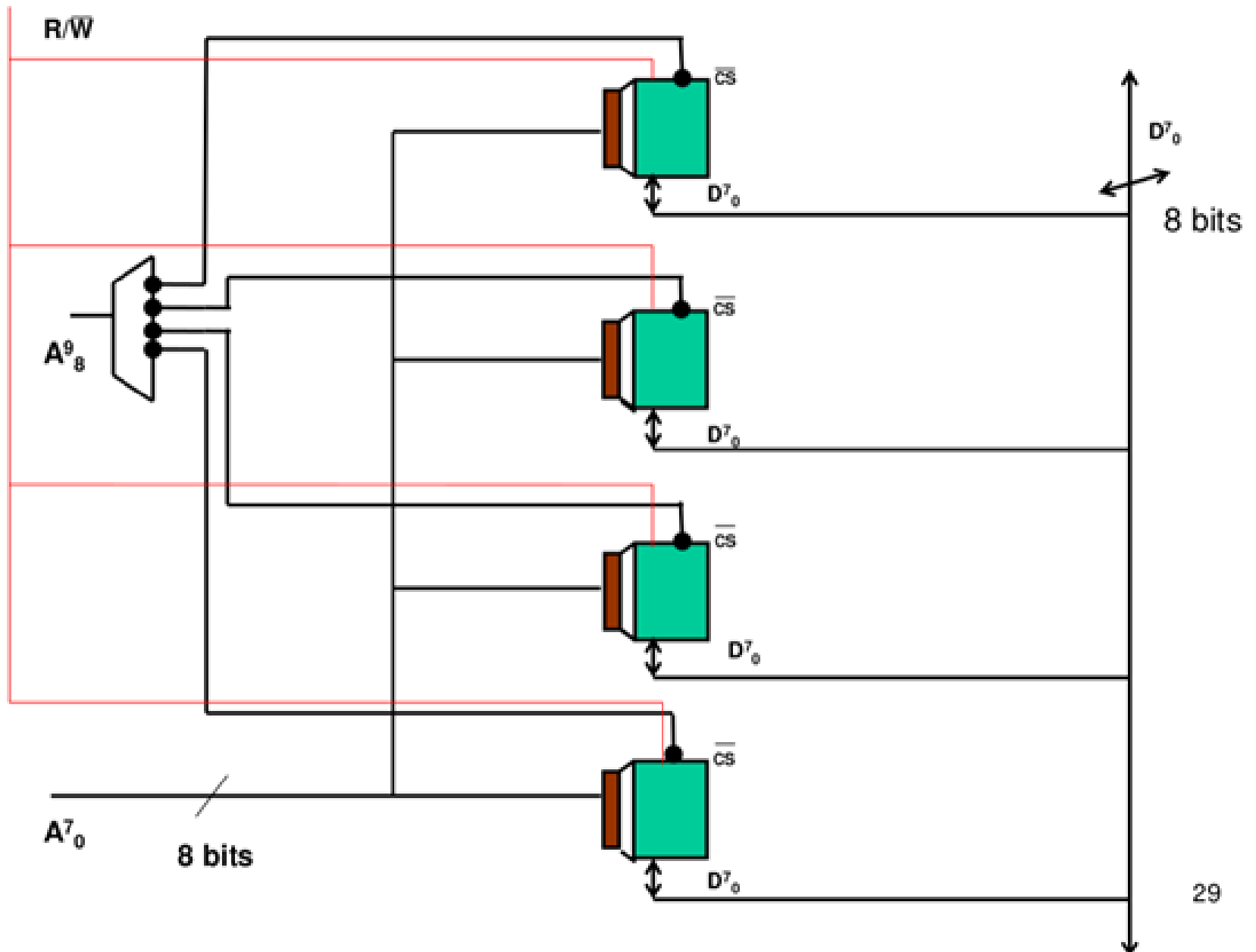
(m',n')=(256,8) correspond à une taille du bus d'adresses de 8 bits (A7'...A0'), taille du bus de données est de 8 bits (D7'....D0')

Calculer les deux facteurs d'extension lignes et colonnes :

$$P = m/m' = 1024/256 = 4 \text{ (extension lignes)}$$

$$Q = n/n' = 8/8 = 1 \text{ (extension colonnes) :}$$

Le nombre totale de boîtiers $P.Q=4$



Microprocesseur

Toutes les informations transitent par le microprocesseur, il exécute un programme contenu en mémoire. Ce **programme est constitué d'un ensemble d'instructions élémentaires codées, qui seront décodées puis exécutées au fur et à mesure par le microprocesseur.**

Un microprocesseur est construit autour de deux éléments principaux :

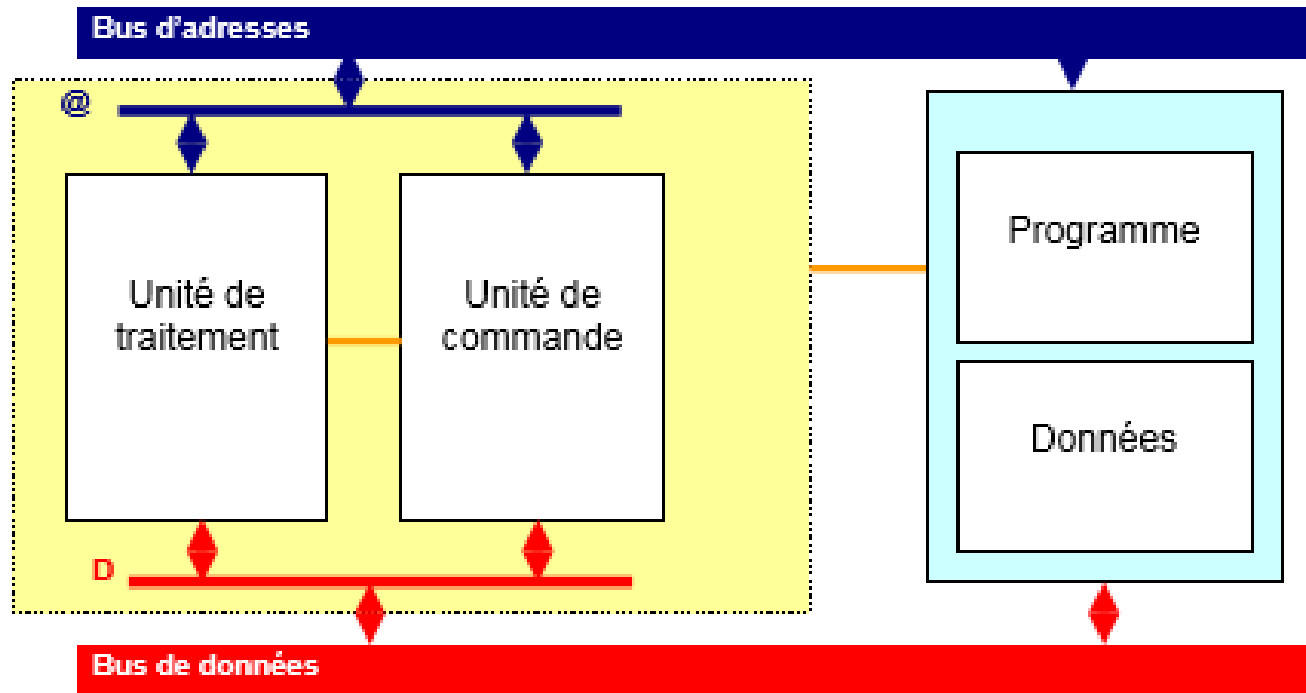
1- Une unité de commande

2- Une unité de traitement

Ces deux éléments sont associés à **des registres** chargés de stocker les différentes informations à traiter. Les trois éléments sont reliés entre eux par **des bus internes** permettant les échanges d'informations.

Il existe deux types de registres :

- **les registres d'usage générale** permettent à l'unité de traitement de manipuler des données à vitesse élevée. Ils sont connectés aux bus de données interne au microprocesseur.
- **les registres d'adresses (pointeurs)** connectés sur le bus d'adresses.



Unité de commande

Elle permet de séquencer le déroulement des instructions. Elle effectue la recherche en mémoire de l'instruction. Comme chaque instruction est codée sous forme binaire, elle en assure le décodage pour enfin réaliser son exécution puis effectue la préparation de l'instruction suivante. Pour cela, elle est composée par : le compteur de programme constitué par un registre dont le contenu est initialisé avec l'adresse de la première instruction du programme. Il contient toujours l'adresse de l'instruction à exécuter.

- **le registre d'instruction et le décodeur d'instruction :**

chacune des instructions à exécuter est rangée dans le registre instruction puis est décodée par le décodeur d'instruction.

- **Bloc logique de commande (ou séquenceur) :**

Il organise l'exécution des instructions au rythme d'une horloge. Il élabore tous les signaux de synchronisation internes ou externes (bus de commande) du microprocesseur en fonction des divers signaux de commande provenant du décodeur d'instruction ou du registre d'état

Unité de traitement

Elle regroupe les circuits qui assurent les traitements nécessaires à l'exécution des instructions :

■ Le registre d'état est généralement composé de 8 bits à considérer individuellement. Chacun de ces bits est un indicateur dont l'état dépend du résultat de la dernière opération effectuée par l'UAL. On les appelle **indicateur d'état** ou **flag ou drapeaux**. Dans un programme le résultat du test de leur état conditionne souvent le déroulement de la suite du programme.

■ On peut citer par exemple les **indicateurs du registre d'état** :

- retenue (carry : C)
- retenue intermédiaire (Auxiliary-Carry : AC)
- signe (Sign : S)
- débordement (overflow : OV ou V)
- zéro (Z)
- parité (Parity : P)

■ Les accumulateurs sont des registres de travail qui servent à stocker un opérande au début d'une opération arithmétique et le résultat à la fin de l'opération.

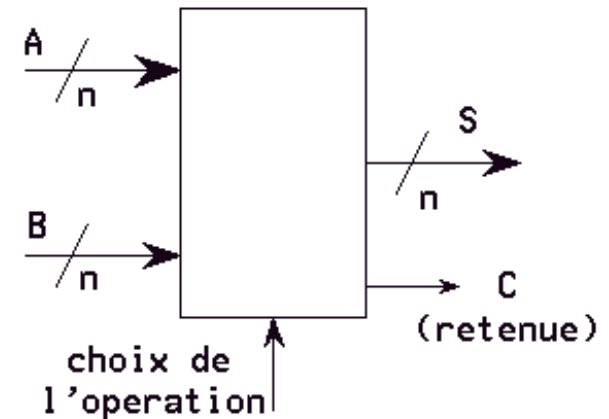
■ L'Unité Arithmétique et Logique (UAL) est un circuit complexe qui assure les fonctions logiques (ET, OU, Comparaison, Décalage , etc...) ou arithmétique (Addition, soustraction, etc...).

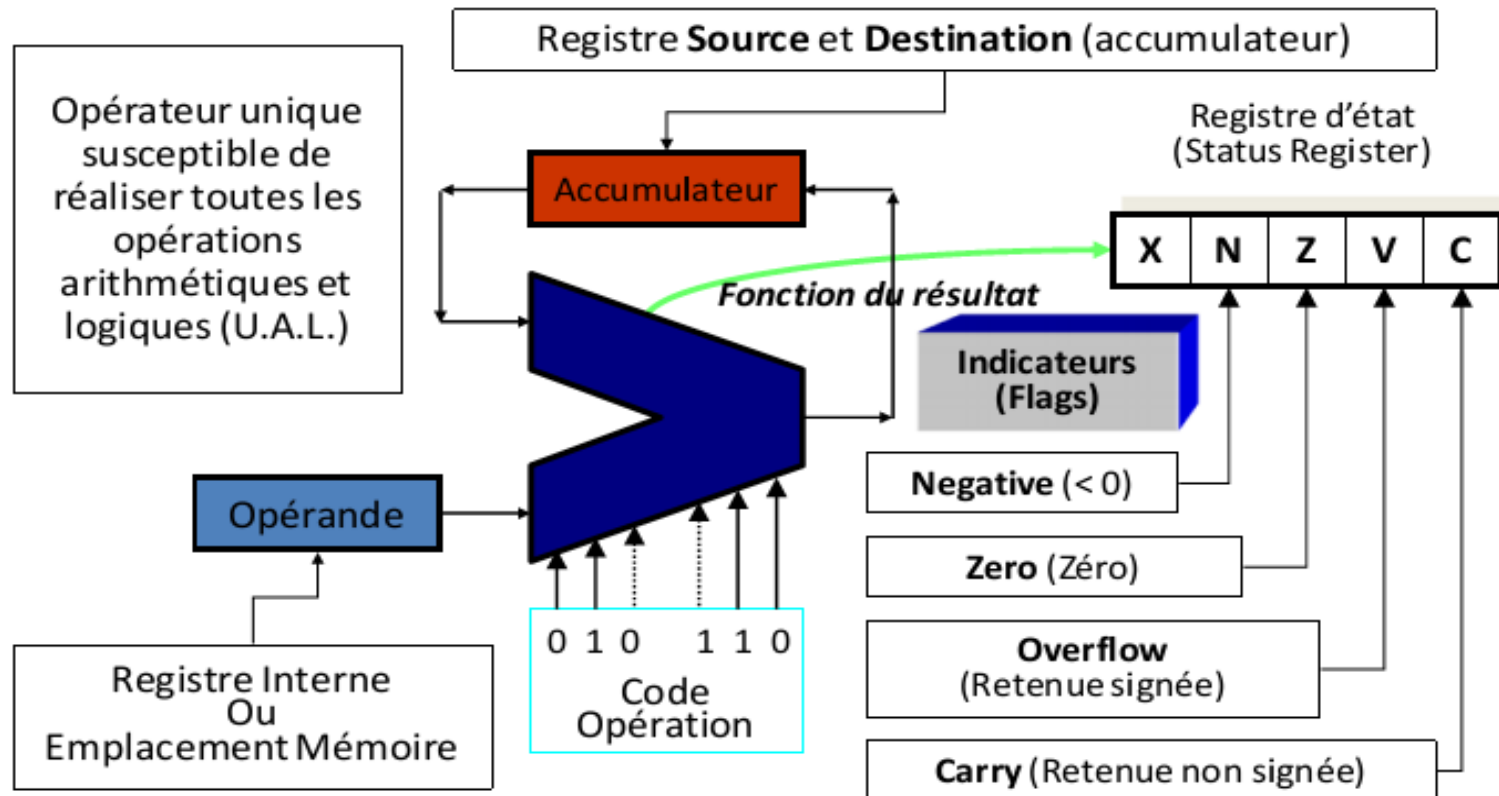
Elle se rapproche **d'un additionneur**, sauf qu'elle est capable en plus de l'addition d'effectuer de nombreuses autres opérations arithmétiques (soustraction, multiplication par 2, division par 2, changement de signe...) ou logiques (décalages, rotations, complémentation, "ou" bit à bit, "et" bit à bit, "ou" exclusif...).

Une UAL ou **ALU** (Arithmétique and Logic Unit) possède donc deux mots de N bits en entrée, un mot de N bits pour le résultat, plus une entrée et une sortie pour la retenue. De plus, d'autres lignes d'entrée permettent la sélection de l'opération désirée :

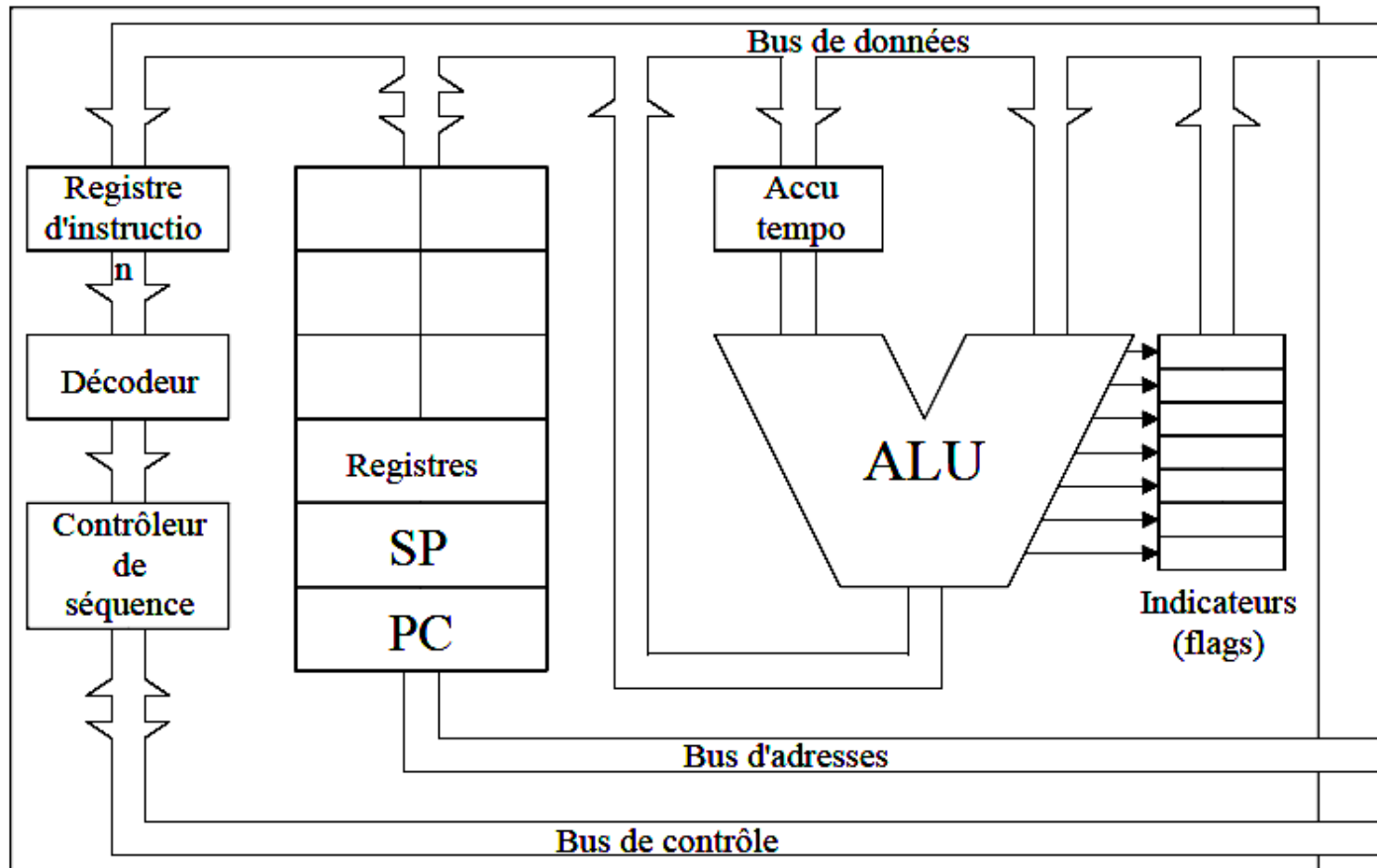
les signaux F_1F_0 permettent de choisir l'opération désirée : par le biais du décodeur, la sortie S est choisie comme provenant de la porte "et", de la porte "ou", de l'inverseur ou de l'additionneur.

F_1	F_0	Opération
0	0	$S = A \text{ ET } B$
0	1	$S = A \text{ OU } B$
1	0	$S = \text{NON } A$
1	1	$S = A + B + C_{in}$





Architecture de base détaillée d'un microprocesseur



En résumé

L'unité centrale d'un microprocesseur comprend essentiellement :

- **une unité arithmétique et logique (UAL)** qui effectue les opérations,
- **des registres** qui permettent au microprocesseur de stocker temporairement des données, tel que l'accumulateur
- **une unité de contrôle** qui commande l'ensemble du microprocesseur en fonction des instructions du programme. Certains registres ont un rôle très particulier
- **le registre indicateur d'état (flags)**, ce registre donne l'état du microprocesseur à tout moment, il peut seulement être lu,
- **le compteur de programme (PC, Program Counter)**, il contient l'adresse de la prochaine instruction à exécuter,
- **le pointeur de pile (SP, Stack Pointer)**, c'est le pointeur d'une zone spéciale de la mémoire appelé pile.

L'unité de contrôle peut aussi se décomposer :

- **le registre d'instruction**, mémorise le code de l'instruction à exécuter,
- **le décodeur** décode cette adresse,
- **le séquenceur** exécute l'instruction, c'est lui qui commande l'ensemble des organes du microprocesseur.

Un système à microprocesseur permet :

- d'acquérir des entrées logiques et analogiques représentant l'état du système technique,
 - d'interpréter, la signification de ces entrées,
 - de calculer, mémoriser, récupérer des variables logicielles intermédiaires, de gérer le temps,
 - d'agir sur des sorties logiques et analogiques en fonction des entrées et des calculs réalisés de manière à modifier le fonctionnement du système technique (commande moteur, affichage d'informations,...),
 - de communiquer par des liaisons séries avec d'autres systèmes techniques et/ou un ordinateur,

Processeurs spéciaux : Microcontrôleur

- Ce sont des **systèmes à microprocesseurs minimum sur une seule puce**.
- Ils contiennent **un CPU, de la RAM, de la ROM et des ports d'Entrée/Sorties** (parallèles, séries, I2C, etc..).
- Ils comportent parfois aussi des fonctions spécifiques comme des compteurs programmables pour effectuer des mesures de durées, **des CAN voir des CNA** pour s'insérer au sein de chaînes d'acquisition, des interfaces pour réseaux de terrain, etc ...
- Il est adapté pour répondre au mieux aux besoin des **systèmes embarquées** (appareil électroménagers, chaîne d'acquisition, lecteur carte à puce, etc...).
- Il est par contre généralement moins puissant en terme de rapidité, de taille de données traitables ou de taille de mémoire adressable qu'un microprocesseur.

Les mémoires du microcontrôleur :

Il existe différents types de mémoires :

EEPROM programme : c'est une mémoire morte dans laquelle on va stocker le programme qui va gérer le fonctionnement du système technique.

EEPROM données : c'est une mémoire morte dans laquelle on va stocker les données devant être sauvegardées si le système technique est mis hors tension.

RAM : mémoire vive dans laquelle on va stocker des données temporaires nécessaires à l'exécution du programme de gestion du système technique. Ces données ne seront plus disponibles si le système technique est mis hors tension

L'horloge du microcontrôleur

Elle va donner la référence temporelle au microprocesseur pour exécuter les instructions.

L'horloge d'un microprocesseur est souvent réalisée grâce à un Quartz.

Il existe certains microcontrôleurs qui ont la possibilité de sélectionner une horloge interne (sans composants externes) ce qui permet d'utiliser les broches de l'horloge pour d'autres périphériques

Les périphériques d'un microcontrôleur

- **Les CAN convertisseure Analogique Numerique** : ce périphérique se trouve souvent implémenté dans le microcontrôleur, il permet d'**acquérir** des grandeurs électriques de type analogique directement à partir d'une ou plusieurs broches du microcontrôleur, la sortie est un nombre binaire.
- **Les CNA : Convertisseur Numerique Analogique** ce périphérique permet de **produire** une tension analogique à partir de mots numériques internes au microcontrôleur.
- **Les ports d'entrées/sorties d'un microcontrôleur** :

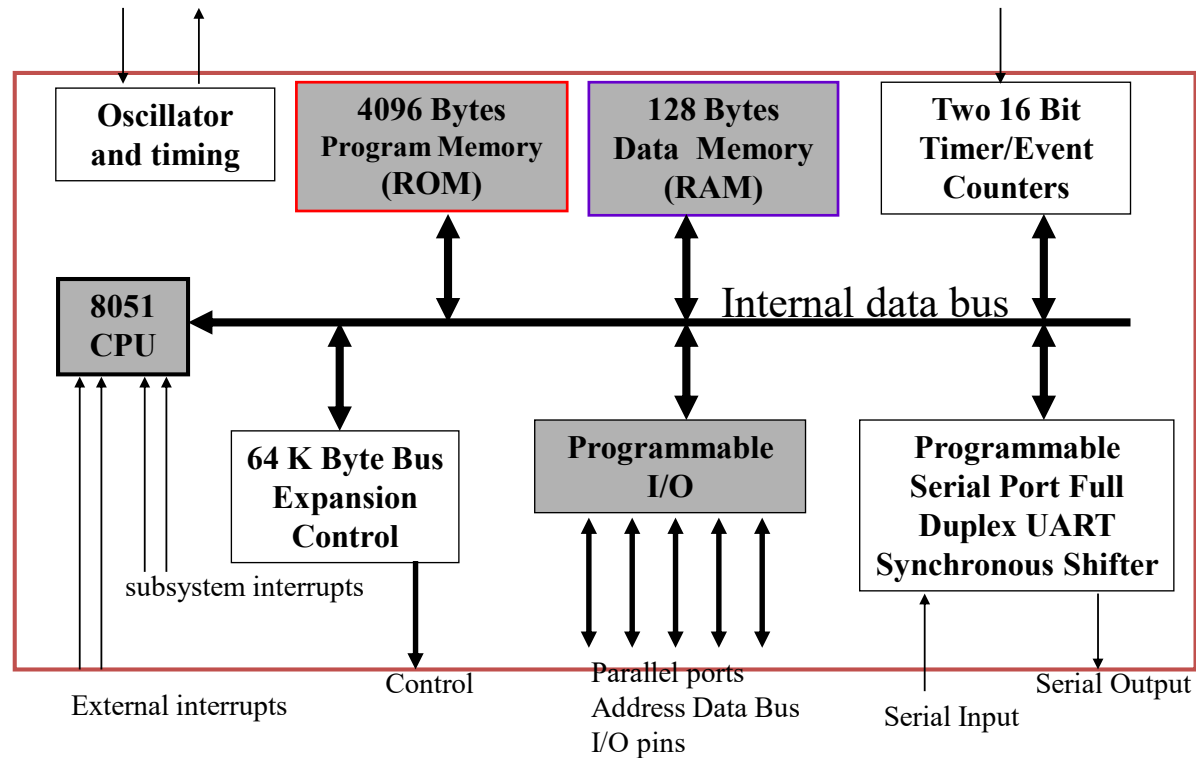
Ces périphériques sont indispensables au microcontrôleur ils permettent :

-d'**acquérir** les entrées de types logiques indiquant l'état du système technique,
-de **produire** des sorties de types logiques permettant de **commander** les périphériques du système techniques (afficheurs, moteurs, buzzer,...)

- **La transmission de données séries asynchrone et synchrone** :

Ces périphériques permettent la communication avec d'autres systèmes technique et/ou un PC

Architecture of 8051 Microcontroller



51

Jeu d'instructions

Le jeu d'instructions décrit l'ensemble des instructions que le microprocesseur pourra exécuter

Les instructions que l'on retrouve dans chaque microprocesseur peuvent être classées en 4 groupes:

- 1 **Transfert de données** pour charger ou sauver en mémoire, effectuer des transferts de registre à registre, etc. . .
- 2 **Opérations arithmétiques** : addition, soustraction, division, multiplication
- 3 **Opérations logiques** : ET, OU, NON, NAND, comparaison, test, etc. . .
- 4 **Contrôle de séquence** : branchement, test, etc. . .

Les **instructions** et leurs **opérandes** (paramètres) sont stockés en mémoire de programme.

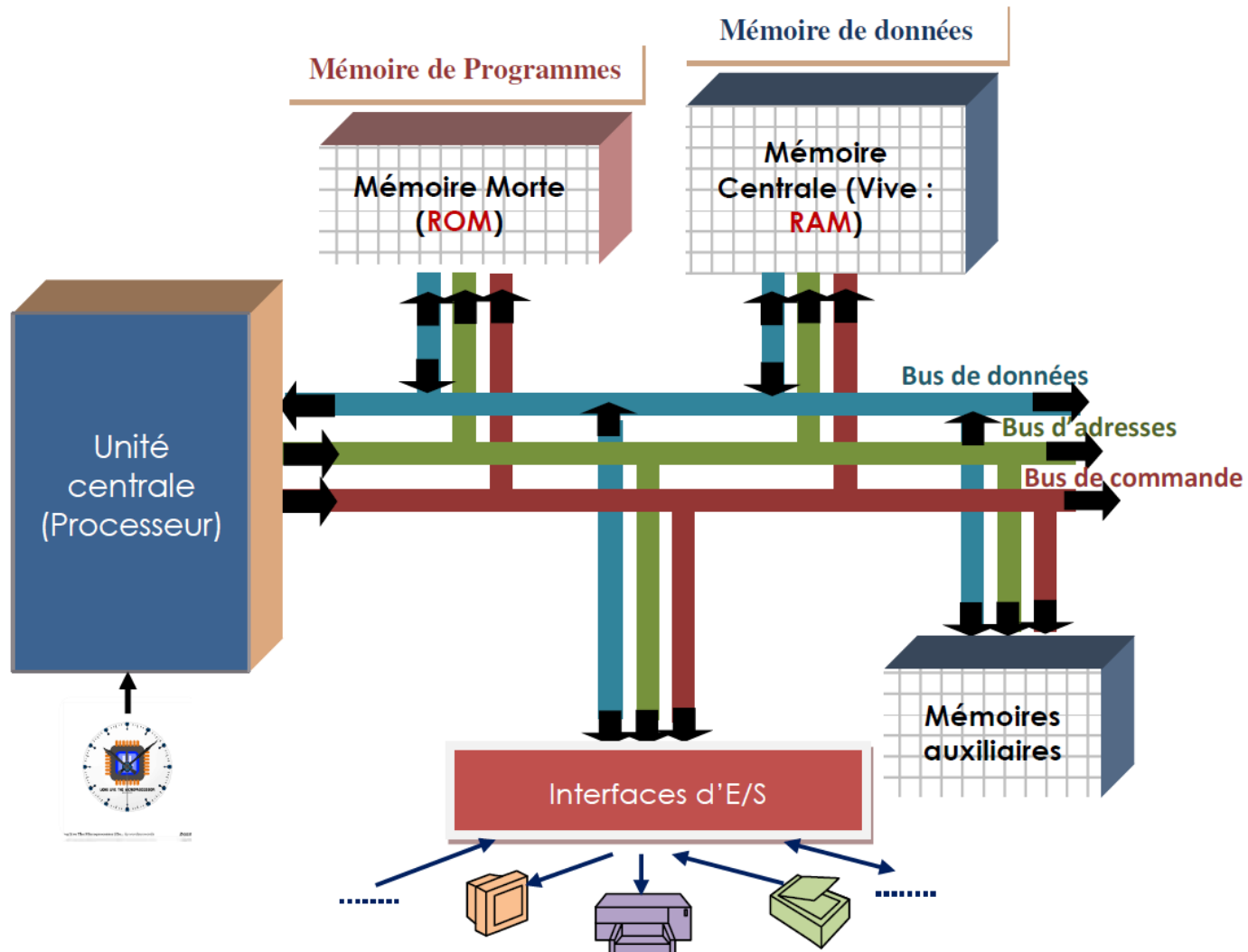
La taille totale d'une instruction (nombre de bits nécessaires pour la représenter en mémoire) dépend du type d'instruction et aussi du type d'opérande. Chaque instruction est toujours codée sur un nombre entier d'octets afin de faciliter son décodage par le processeur. Une instruction est composée de deux champs :

- **le code instruction**, qui indique au processeur quelle instruction réaliser
- **le champ opérande** qui contient la donnée, ou la référence à une donnée en mémoire (son adresse).

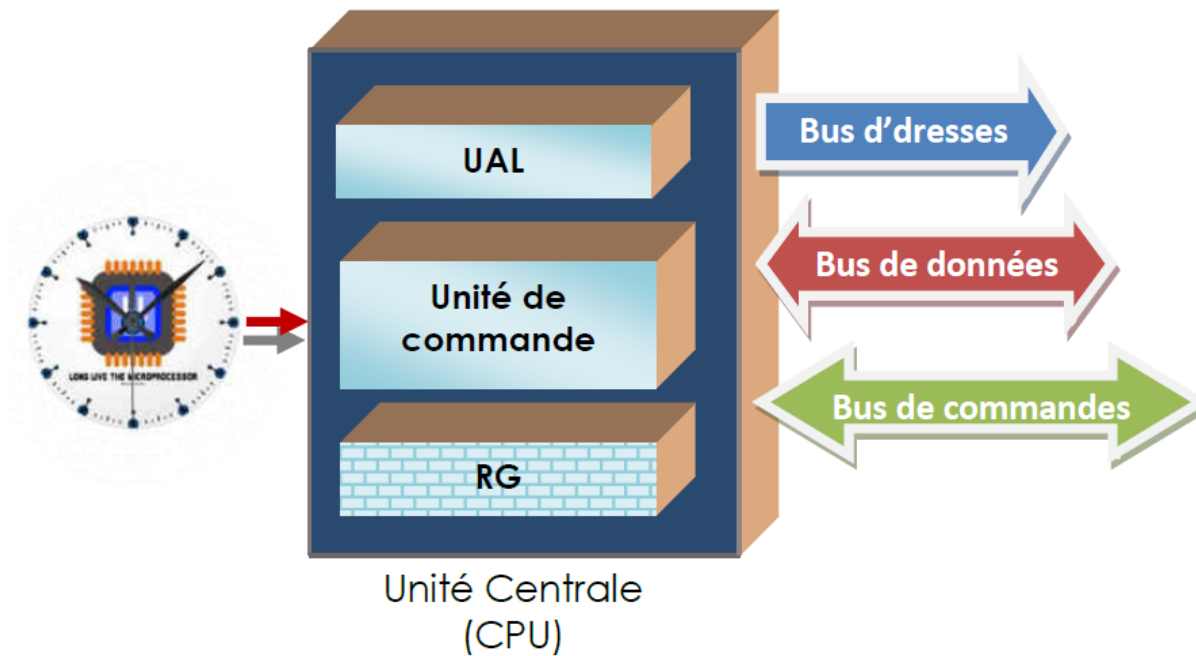
■ Exemple :

Code instruction	Code opérande
1001 0011	0011 1110

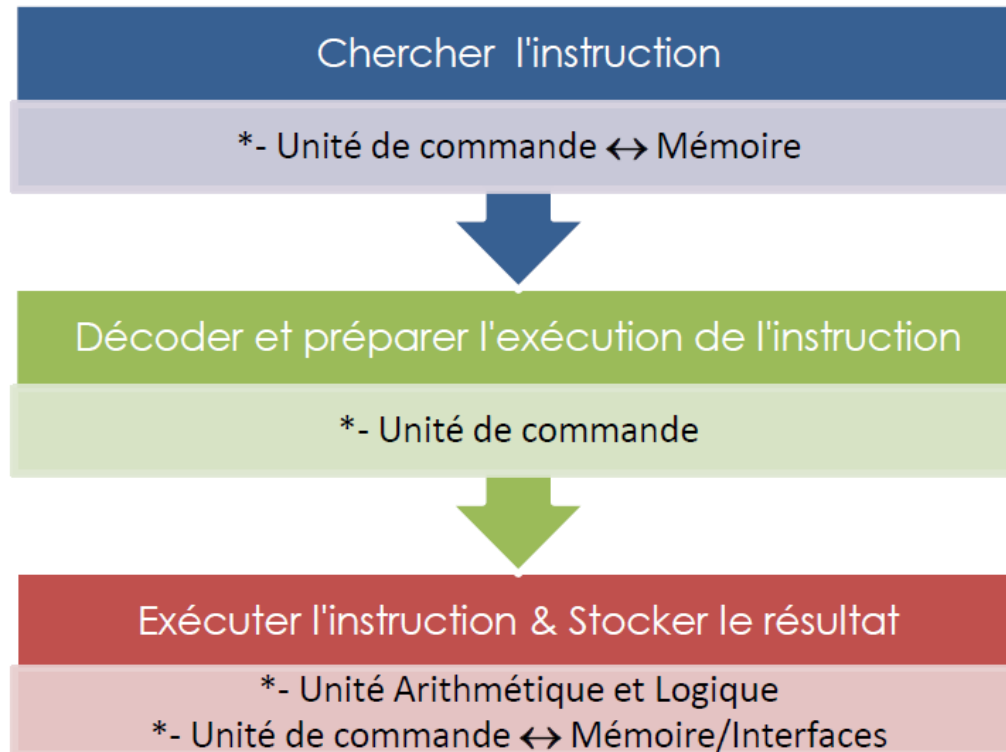
Architecture schématique d'un système microprogrammé



Structure de l'unité centrale de traitement



Etapes d'exécution d'une instruction dans un système microprogrammé (embarqué)

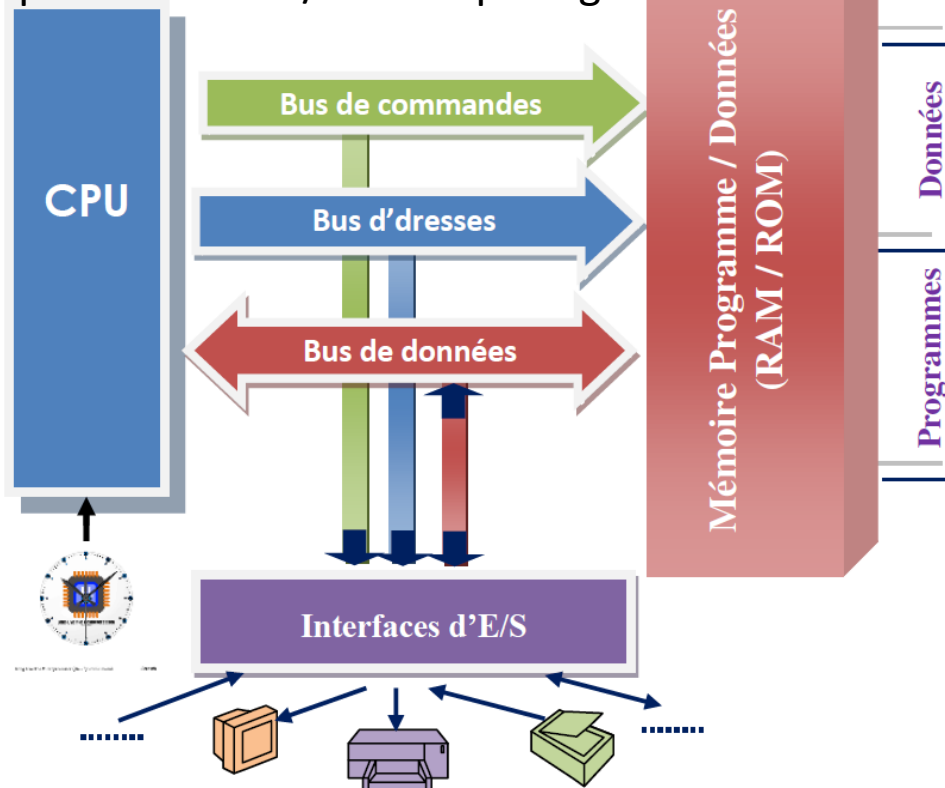


Architectures de Von Neumann et de Harvard

Pour l'organisation des différents blocs constituant un système embarqué minimum, il existe deux architectures possibles:

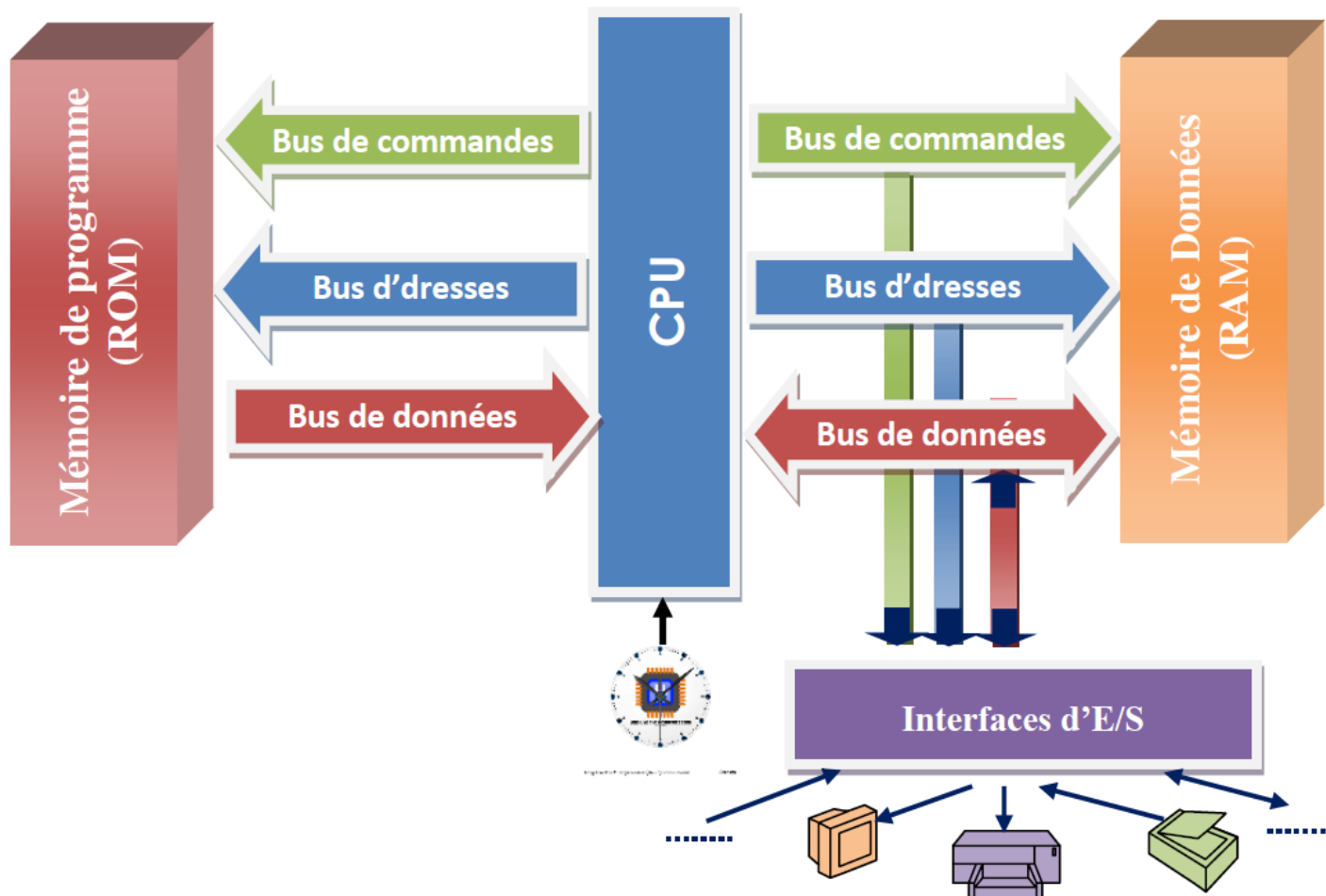
- Architecture de Von Neumann.
- Architecture de de Harvard.

Dans l'architecture de **Von Neumann**, la mémoire programme, la mémoire données et les périphériques d'entrées/sorties partagent le même bus s'adressant et de données.



Architectures de Von Neumann et de Harvard

L'architecture de type **Harvard** est une conception de systèmes embarqués minimum qui sépare physiquement la mémoire de données et la mémoire programme. L'accès à chaque type de mémoires s'effectue via ses propres bus.



Processeurs de types CISC et RISC

Actuellement l'architecture des microprocesseurs se composent de deux grandes familles :

- Processeurs à jeu d'instruction étendu ou **CISC** (Complex Instruction Set Computing): Elle consiste à rajouter autant d'instructions que possible au système microprogrammé (processeur).
- Processeurs à jeu d'instructions réduit ou **RISC** (Reduced Instruction Set Computing): Elle consiste à minimiser le nombre d'instructions et à les simplifier dans un système microprogrammé (processeur).

L'architecture RISC a montré qu'elle était plus performante :

- Tous les processeurs fonctionnent en RISC de nos jours.
- Pour un processeur avec un jeu d'instruction CISC: Traduction interne des instructions CISC en micro-opérations de type RISC.

Processeurs de types CISC et RISC

Deux paramètres sont souvent utilisés pour mesurer la performance d'un processeur :

- Latence: Le temps de réponse (temps d'exécution d'une instruction) : Le temps qui s'écoule entre le début et la fin d'exécution de l'instruction.
- Débit: Quantité total de travail réalisé en fonction du temps : Nombre d'opérations (tâche, instruction) exécutées par unités de temps.

*L'amélioration du temps de réponse implique toujours une amélioration du débit.
Par contre, le contraire n'est toujours vrai.*

Par exemple, augmenter le nombre de processus (instruction) augmentera le débit mais pas le temps d'exécution.

Notions de pipeline

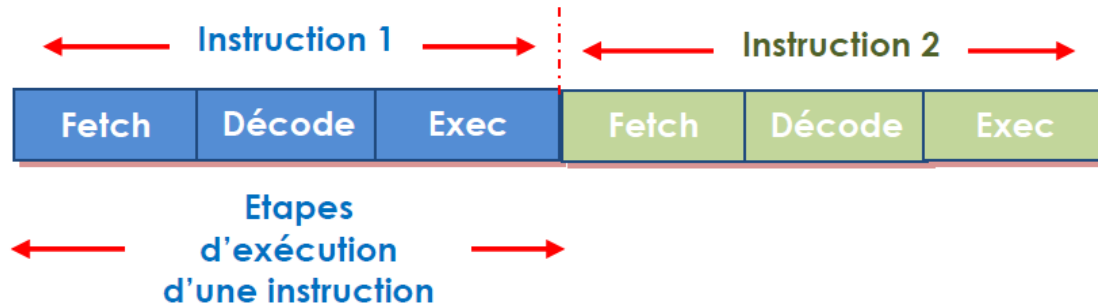
La notion de pipeline, autrement dit la chaîne de traitement des instructions, est l'élément d'un processeur dans lequel l'exécution des instructions est découpée en plusieurs étapes, c'est-à dire que le processeur peut commencer à exécuter une nouvelle instruction sans attendre que la précédente soit terminée. C'est une sorte d'exécution parallèle des instructions (Parallélisme).

- Le pipeline est une technique utilisée pour optimiser le temps d'exécution d'un programme contenant plusieurs instructions (augmentation du nombre de tâches exécutées par unité de temps).
- Chacune des étapes du pipeline est implémentée par un circuit intégré indépendant, appelé **étage**.
- Le nombre d'étages d'un pipeline est appelé sa **profondeur**.

Notions de pipeline

Processeur sans pipeline :

Dans ce type de processeur, le traitement de l'ensemble des instructions (processus) s'effectue d'une manière normale non pipeliné ou séquentiel : Les instructions sont traitées les unes à la suite des autres.



- ✓ Phase A = Cycle de recherche d'instruction (**Fetch** (IF));
- ✓ Phase B = Cycle de décodage et préparation de l'exécution (**Decode** (ID));
- ✓ Phase C = Cycle d'exécution effective de l'instruction (**Exec** (IE)) ;

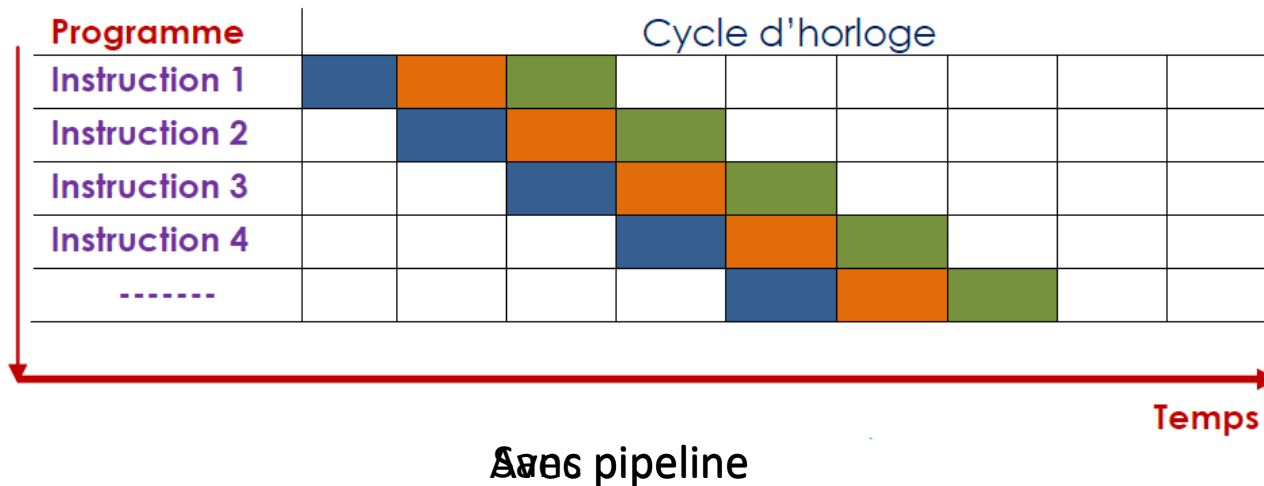
→ Cycle machine = phase A + phase B + phase C = Temps d'exécution d'une instruction ;

Notions de pipeline

Processeur avec pipeline :

Pour optimiser le temps d'exécution d'un programme contenant plusieurs instructions (processus) et d'augmenter le nombre de tâches exécutées par unité de temps, la technique de pipeline est utilisée.

- A chaque étape du processus est affectée une ressource indépendante, de façon à pouvoir exécuter plusieurs processus en parallèle où chacun à une étape différente.
- Exécution simultanée de différentes étapes sur des données distinctes.



Microprocesseur ou microcontrôleur ?

Les microprocesseurs et les microcontrôleurs sont des puces électroniques programmables typiques utilisées à des fins différentes.

- La différence principale entre eux est qu'un microprocesseur est un moteur de calcul programmable constitué d'une unité arithmétique et logique, d'un processeur et de registres, capable d'effectuer des calculs et de prendre des décisions.
- Par contre, un microcontrôleur est un microprocesseur spécialisé considéré comme un ordinateur sur une puce (*System On Chip*). Il contient
 - Un CPU,
 - De la RAM (mémoire de données),
 - De la ROM (mémoire programme) et
 - Des ports d'E/S.

Il comporte aussi des fonctions spécifiques (modules annexes) comme

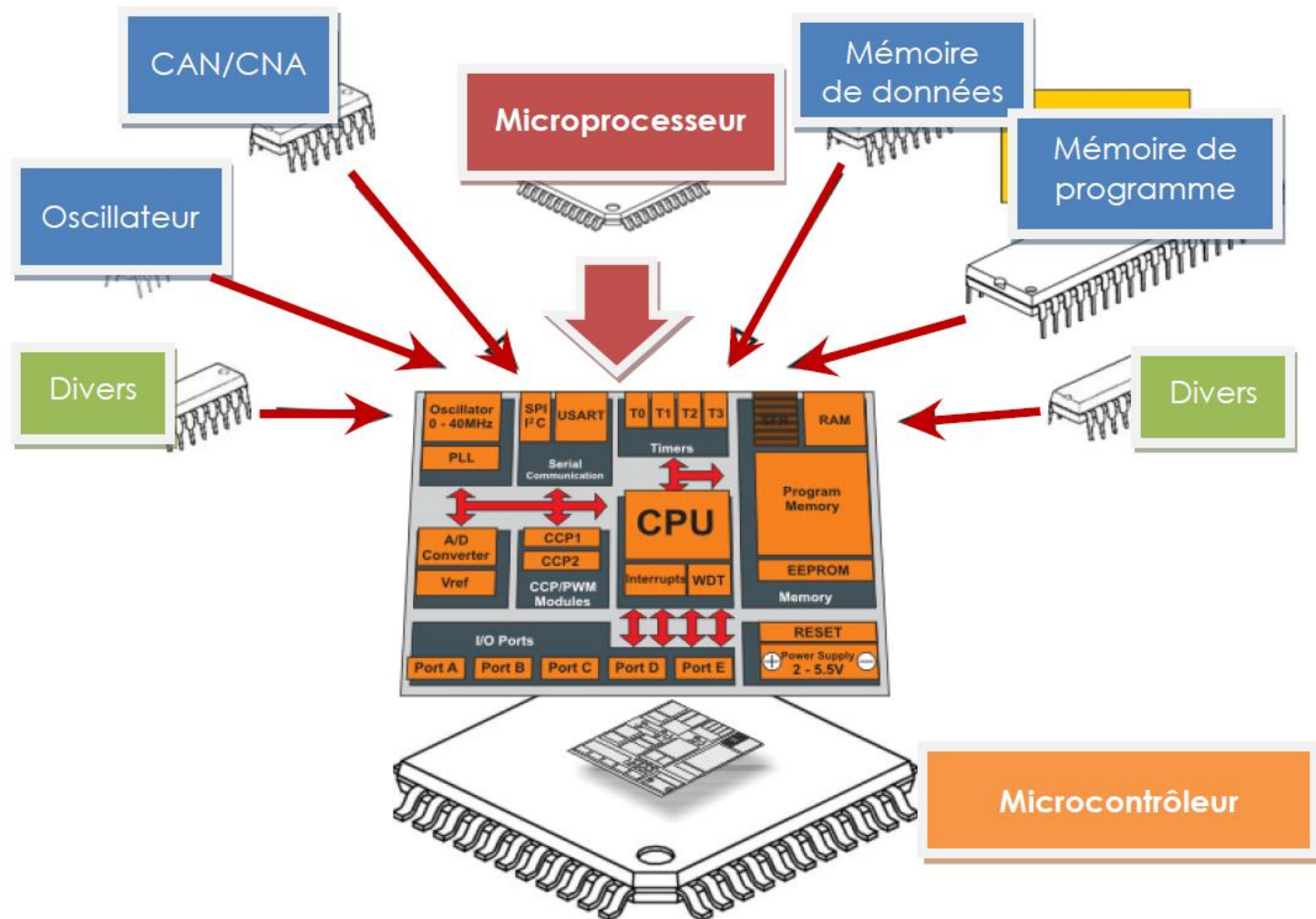
- Des compteurs programmables pour effectuer des mesures de durées (Timers),
- Des convertisseurs analogiques/ numériques (CAN) (voir des CNA), etc.

Microcontrôleur = Microprocesseur + RAM + ROM + Interfaces E/S

Circuits intégrés dans le même boîtier

Microprocesseur ou microcontrôleur ?

Différences structurelles entre microprocesseur et microcontrôleur



Microprocesseur ou microcontrôleur ?

Différences structurelles entre microprocesseur et microcontrôleur

Carte mère – Ordinateur

(Microprocesseur + Mémoires +
Circuits électroniques)



Microcontrôleur

Regroupe dans une seule puce
(Microprocesseur + Mémoires + Circuits
électroniques)



Microcontrôleur est un ordinateur
monté dans un circuit intégré.

Microprocesseur	Microcontrôleur
Le microprocesseur est le cœur d'un système informatique.	Le microcontrôleur est le cœur d'un système embarqué (Embedded System).
C'est juste un processeur. Les composants de mémoire et d'E/S doivent être connectés en externe.	Le microcontrôleur dispose d'un processeur ainsi que de composants de mémoire interne et d'E/S.
Ne peut pas être utilisé dans des systèmes compacts et donc inefficace.	Peut être utilisé dans des systèmes compacts et constitue donc une technique efficace.
Les systèmes à microprocesseur sont plutôt réservés pour les applications demandant beaucoup de traitement de l'information et assez peu de gestion d'entrées / sorties. Les ordinateurs sont réalisés avec des microprocesseur.	Les microcontrôleurs sont plutôt dédiés aux applications spécifiques qui ne nécessitent pas une grande quantité de calculs complexes, mais qui demandent beaucoup de manipulations d'entrées/sorties.
Les microprocesseurs sont basés sur l'architecture von Neumann dans laquelle le programme et les données sont stockés dans le même module de mémoire.	Les microcontrôleurs sont basés sur l'architecture de Harvard où la mémoire du programme et la mémoire de données sont séparées.

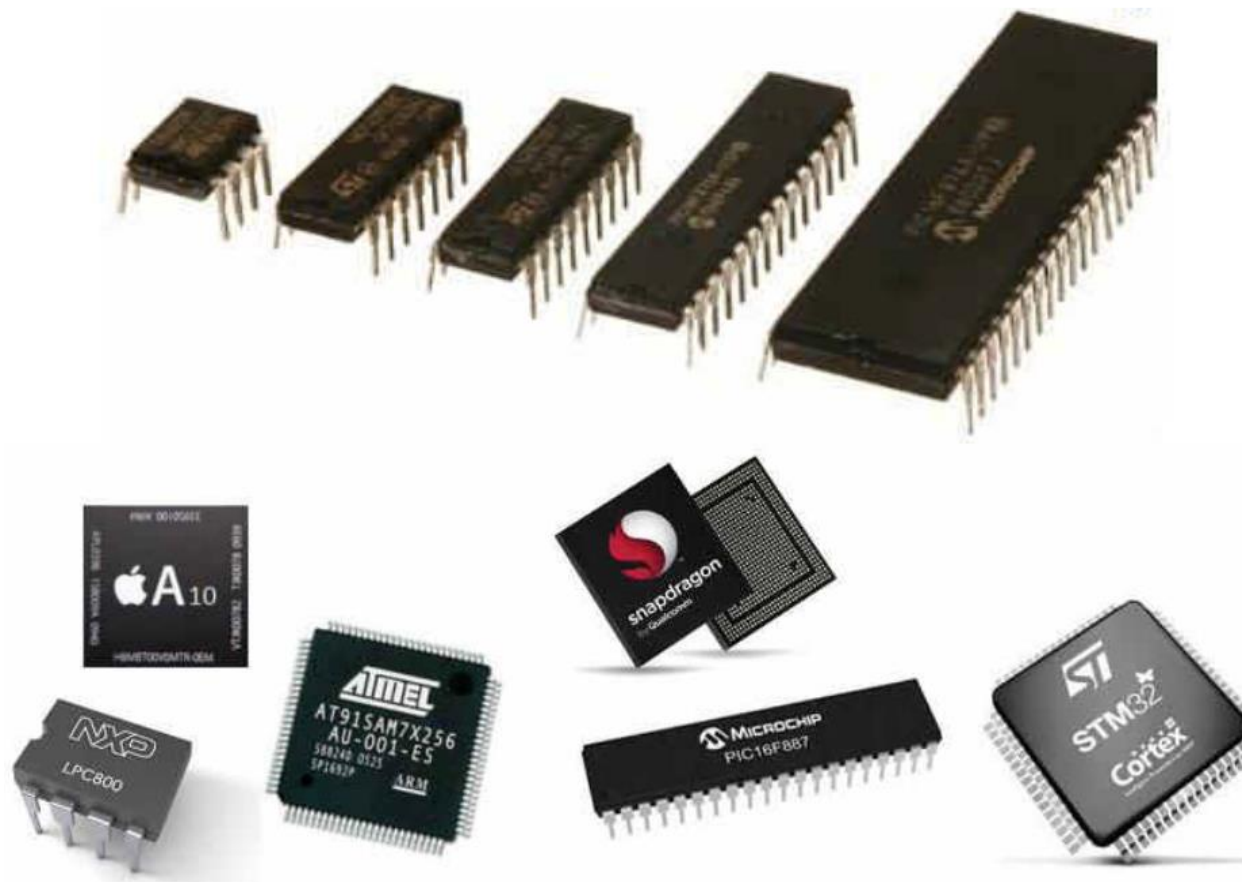
Différentes familles des microcontrôleurs

Les fabricants les plus connus sont mentionnés dans le tableau ci-dessous.

Fabricant	Logo	Série	Famille
Microship	 MICROCHIP	PIC	PIC10xxx, PIC12xxx, PIC16xxx, PIC24xxx, PIC32xxx
Atmel		AVR	Atmega, AT90, AVR32
Intel		MCS-51	80C50, 80C31
Toshiba	TOSHIBA		TX19A

Critères de choix du microcontrôleur

Il existe un très grand nombre de modèles de microcontrôleurs, où de nombreux fabricants proposent chacun plusieurs familles (comptant parfois de centaines de modèles). Ces fabricants proposent aussi des microcontrôleurs allant de petits circuits à 8 pattes (faible prix) jusqu'à des circuits comptant de centaines de pattes.



Critères de choix du microcontrôleur

Critères techniques du choix du microcontrôleur :

- ✓ Nombre de broches du circuit.
- ✓ Taille de la mémoire de programme.
- ✓ Taille de la mémoire de données.
- ✓ Besoin en puissance de calcul : famille 8, 16 ou 32 bits.
- ✓ Type de boîtier et caractéristiques de consommation (Tension d'alimentation, courant de consommation).
- ✓ Périphériques embarqués.
- ✓ Taille de bus de données (capacité de traitement de système, format du mot traité).
- ✓ Taille de bus d'adresses
- ✓ Vitesse d'horloge.

Architecture matérielle du microcontrôleur PIC

Généralement, les PIC (de Microchip) sont caractérisés par:

- ✓ Séparation des mémoires de programmes et de données (Architecture de Harvard).
- ✓ Communication avec l'extérieur seulement via des ports d'entrées/sorties.
- ✓ Utilisation d'un jeu d'instructions réduit RISC (Reduced instruction set computer).

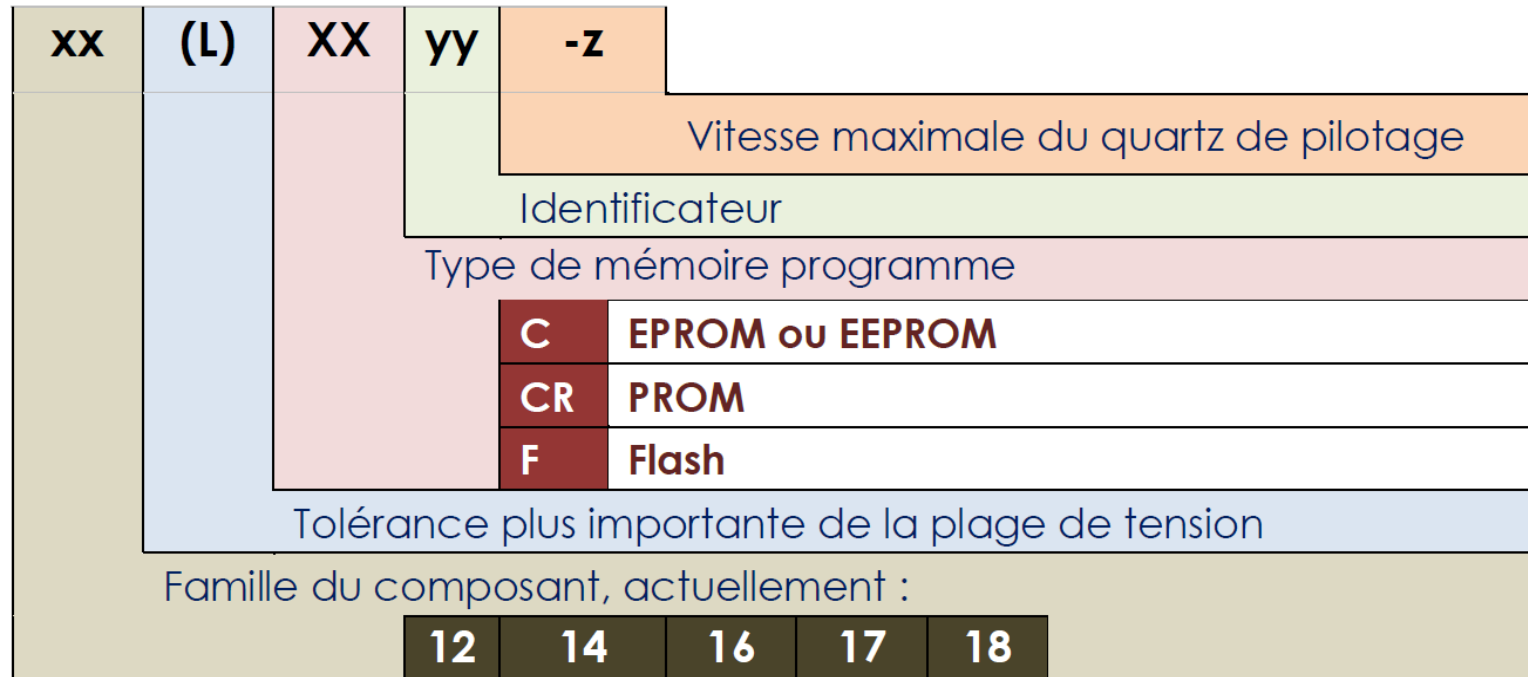
Actuellement les modèles Microchip 8 bits (Tailles des registres internes de données est égale 8 bits), sont classés en 3 grandes familles :

- ✓ Base-Line : Les instructions sont codées sur 12 bits.
- ✓ Mid-Line : Les instructions sont codées sur 14 bits.
- ✓ High-End : Les instructions sont codées sur 16 bits.

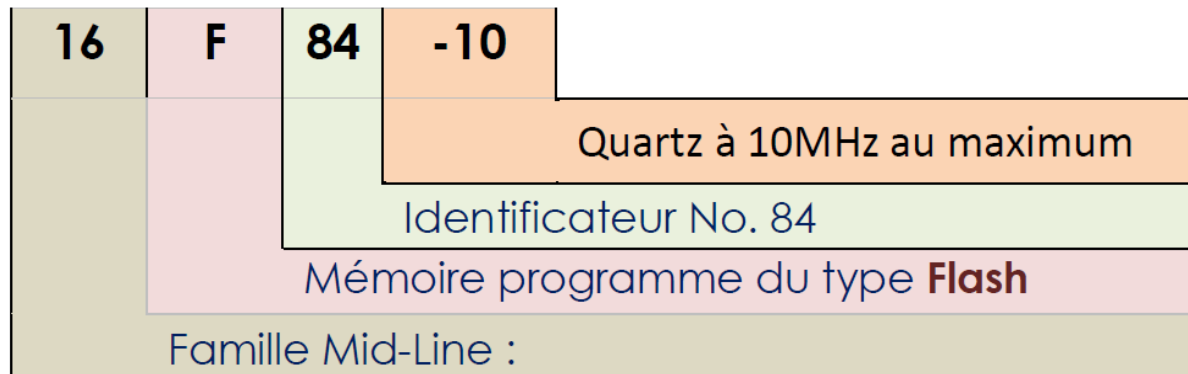
Famille	Taille du mot d'instruction	Nombre d'instructions	Exemples de circuits
Base-Line	12 bits	33	10F200, 12F508, 16F57
Mid-Range	14 bits	35	12F609, 16F84A, 16F631, 16F873A
High-End	16 bits	75	18F242, 18F2420

Architecture matérielle du microcontrôleur PIC

Un PIC est généralement identifié par une référence de la forme suivante :



Exemple :



Microcontrôleur PIC 16F84

Principales caractéristiques du PIC 16F84 :

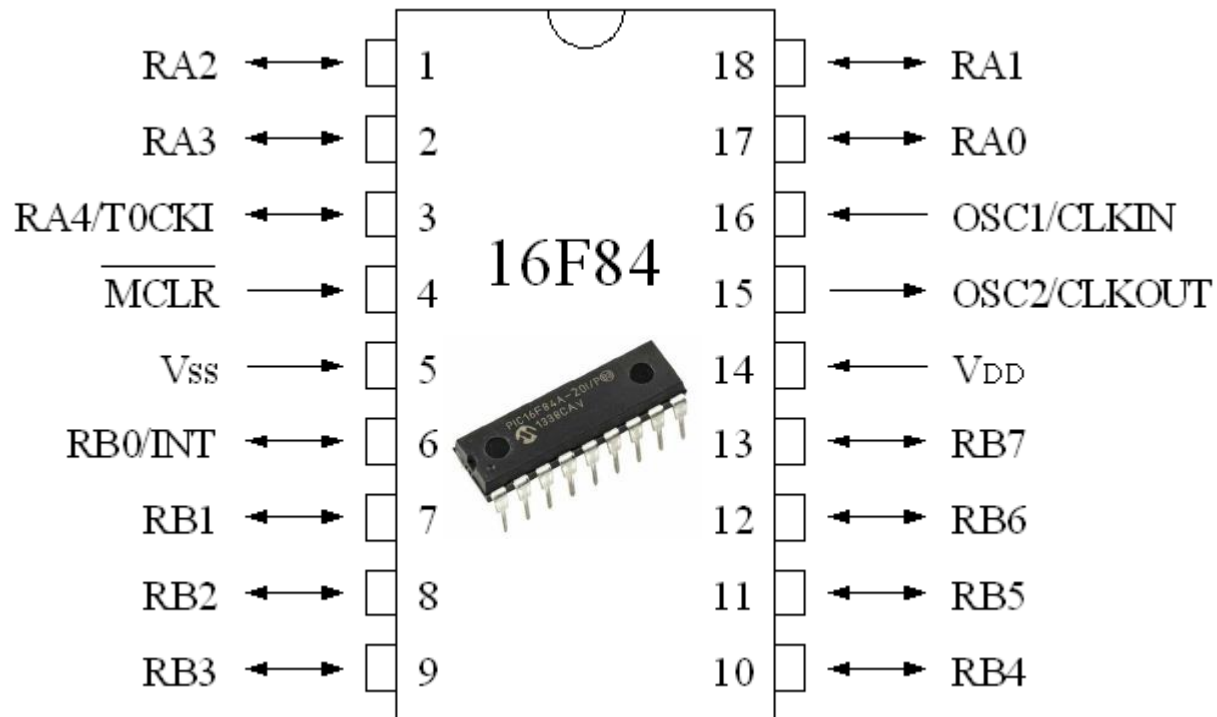
- ✓ 35 Instructions (RISC) codées sur 14 bits (famille Mide-Line).
- ✓ Données sur 8 bits.
- ✓ Vitesse maximum 10 MHz soit une instruction en 400 ns (1 cycle machine = 4 cycles d'horloge).
- ✓ Deux ports d'entrées / sorties.
- ✓ 4 sources d'interruption.
- ✓ 1000 cycles d'effacement/écriture pour la mémoire flash.
- ✓ 10.000.000 pour la mémoire de donnée EEPROM.

Principaux constituants du PIC 16F84 :

- ✓ Mémoire de type FlashROM pour le programme;
- ✓ Mémoire de type RAM pour les registres;
- ✓ Mémoire de type EEPROM pour les données à sauvegarder;
- ✓ Registres particuliers;
- ✓ Unité Arithmétique et Logique (UAL) ;
- ✓ Ports d'entrées / sorties;

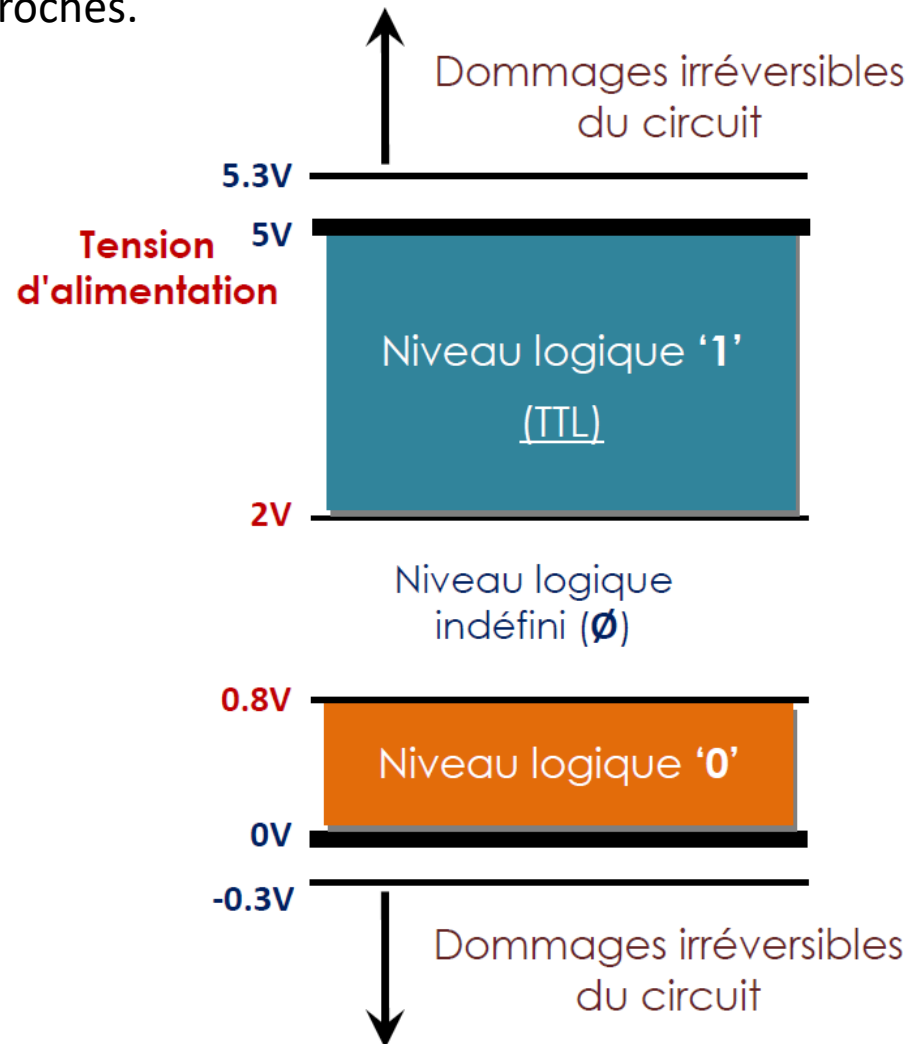
Microcontrôleur PIC 16F84

Ce microcontrôleur est commercialisé dans un boîtier DIP (Plastic Dual In-line Package) en 18 broches.



Microcontrôleur PIC 16F84

Ce microcontrôleur est commercialisé dans un boîtier DIP (Plastic Dual In-line Package) en 18 broches.

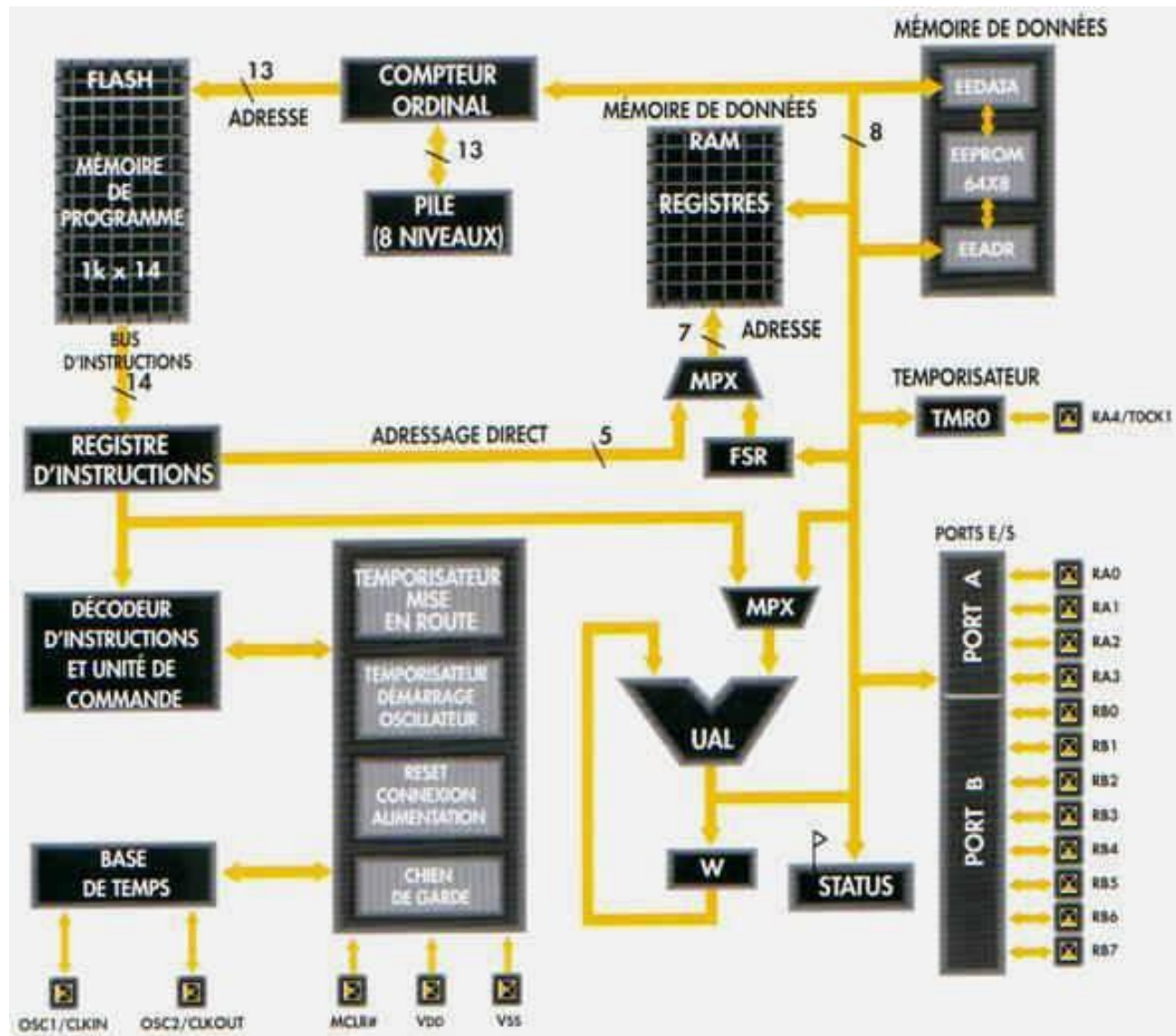


Microcontrôleur PIC 16F84

L'utilisation et la mise en œuvre très simple des PICs les a rendus extrêmement populaire. Pour faire fonctionner le PIC, il suffit :

- ✓ D'alimenter le circuit par ses deux broches VDD et VSS.
- ✓ De fixer sa vitesse de fonctionnement à l'aide d'un QUARTZ.
- ✓ D'élaborer un petit système pour permettre de réinitialiser le microcontrôleur sans avoir à couper l'alimentation (RESET (Master Clear)).
- ✓ D'écrire le programme en langage assembleur ou en C sur un ordinateur puis de le compiler pour le transformer en langage machine et le transférer dans le PIC grâce à un programmeur.
- ✓ De mettre en service l'application développée.

Architecture interne du PIC 16F84.



Fonctionnement du PIC 16F84

Pour mieux comprendre le bon fonctionnement du PIC 16F84, les blocs fondamentaux constituant l'architecture interne seront étudiés en détails, à savoir :

- ✓ L'Horloge.
- ✓ Le RESET.
- ✓ Le processeur ou l'Unité Centrale.
- ✓ Les mémoires (Programme / Données).

Les ressources auxiliaires :

- ✓ Ports d'entrées et de sorties.
- ✓ Temporisateur (TMR).
- ✓ Chien de garde (WATCHDOG).
- ✓ Mode sommeil (consommation réduite).
- ✓ Interruptions.

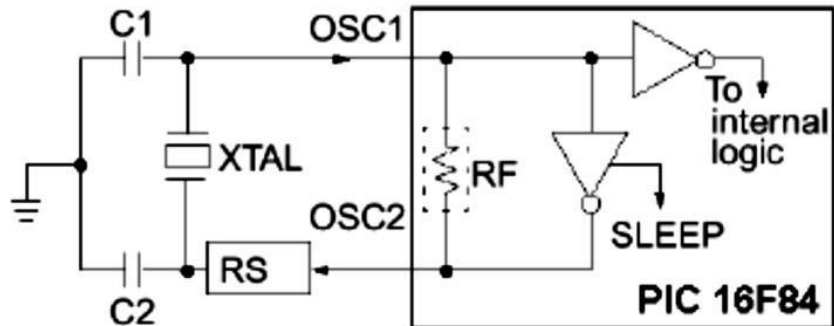
L'Horloge

Les broches OSC1/CLOCKIN et OSC2/CLOCKOUT sont réservées à l'horloge.
L'horloge du PIC16F84A peut être soit interne soit externe.

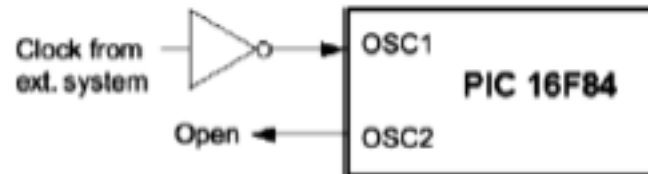
- ✓ L'Horloge interne est constituée d'un oscillateur à quartz ou d'un oscillateur RC.
- ✓ Avec l'oscillateur à Quartz, on peut avoir des fréquences allant jusqu'à 4, 10 ou 20 MHz.
- ✓ Dans certains cas, une horloge externe au microcontrôleur peut être utilisée pour synchroniser le PIC sur un processus particulier.
- ✓ Selon la configuration des bits FOSC1 et FOSC0 du registre de CONFIGURATION, quatre modes de fonctionnement peuvent être définies :
 - ☐ FOSC1=0 et FOSC0=0 → Réseau de R et C (Fmax = 4MHz).
 - ☐ FOSC1=0 et FOSC0=1 → Oscillateur à QUARTZ ou externe (Fmax = 4MHz).
 - ☐ FOSC1=1 et FOSC0=0 → Oscillateur à QUARTZ grand vitesse (Fmax= 10MHz).
 - ☐ FOSC1=1 et FOSC0=1 → Oscillateur à QUARTZ basse consommation (Fmax= 200KHz).

L'Horloge

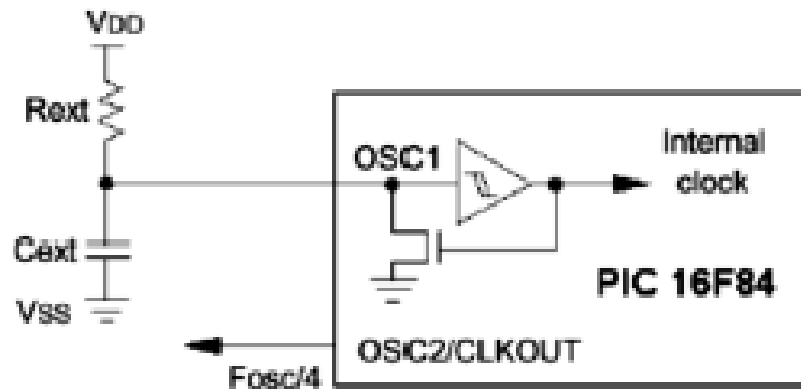
Un quartz ou résonateur céramique



Un oscillateur externe



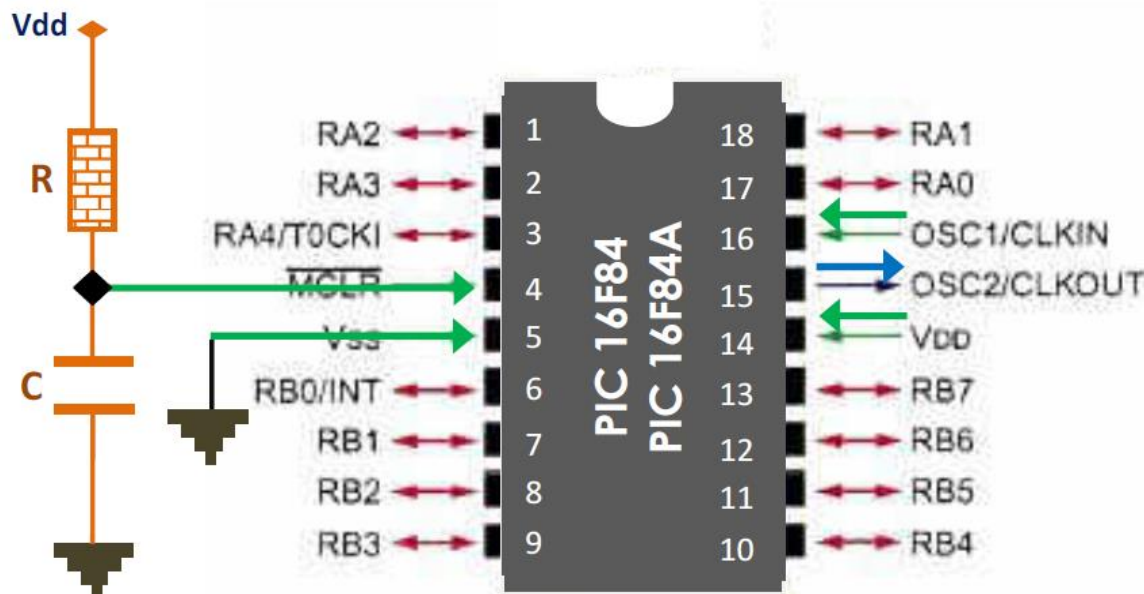
Un réseau RC



Recommended values: $5 \text{ k}\Omega \leq R_{\text{ext}} \leq 100 \text{ k}\Omega$
 $C_{\text{ext}} > 20 \text{ pF}$

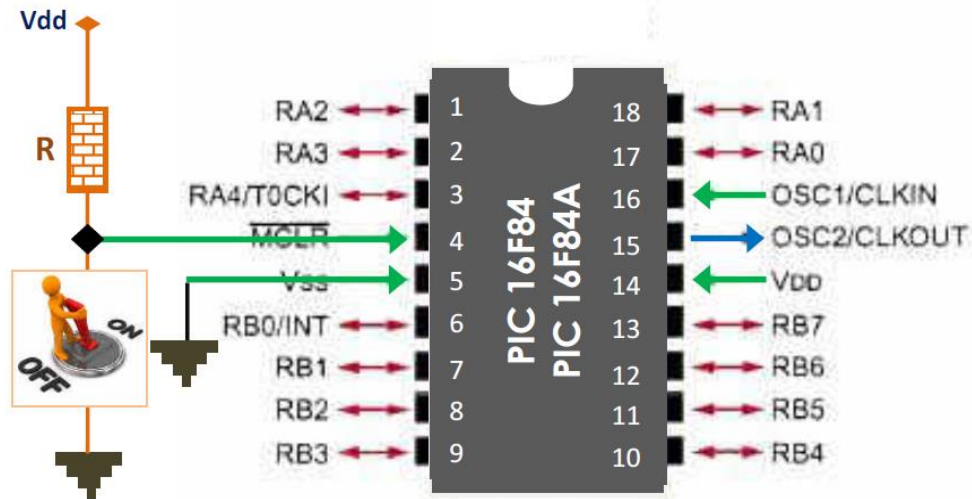
Le RESET

Le RESET ou l'opération de réinitialisation du microcontrôleur se fait généralement via l'entrée MCLR (Broche 4). Le RESET s'effectue en mettant un '0' logique sur la broche 4 du microcontrôleur. Il existe plusieurs matérialisations de système de RESET: RESET manuel, RESET automatique et RESET mixte.

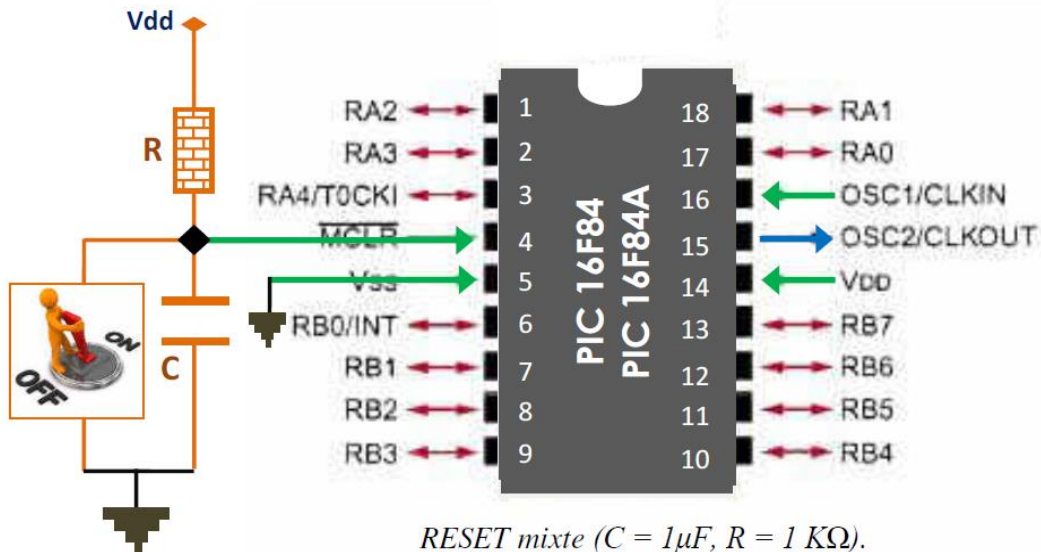


RESET automatique ($C = 1\mu F$, $R = 1 K\Omega$).

Le RESET



RESET Manuel ($R = 1\text{ K}\Omega$).

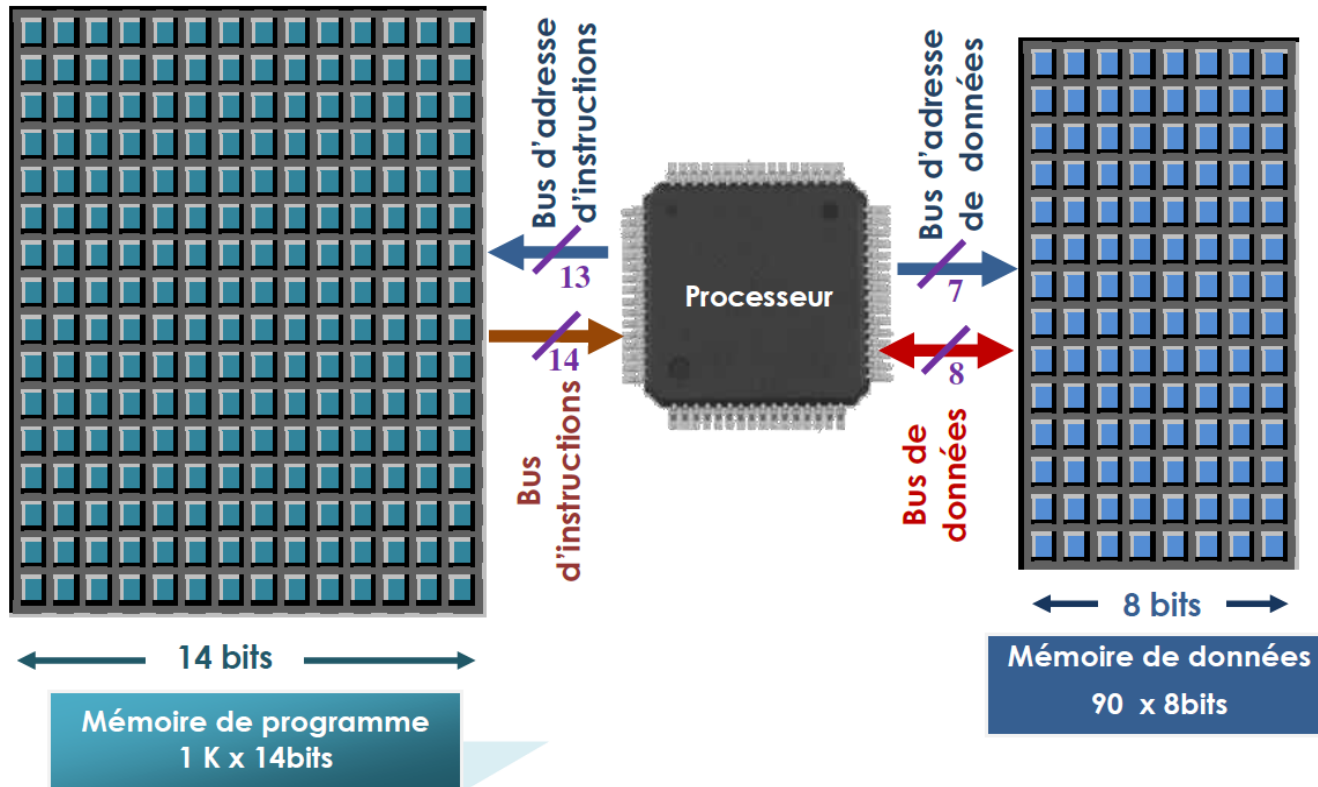


RESET mixte ($C = 1\mu\text{F}$, $R = 1\text{ K}\Omega$).

Organisation de la mémoire

Le PIC 16F84 est conçu selon l'architecture de Harvard. Donc, il possède 2 mémoires:

- ✓ Mémoire de programme.
- ✓ Mémoire des données.

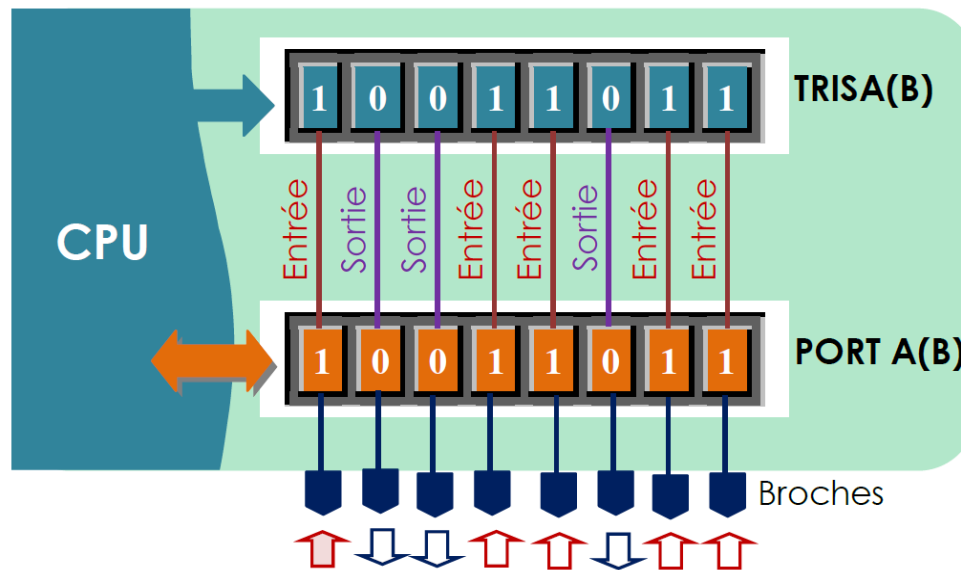


Les ports d'Entrées / Sorties

Pour dialoguer avec l'extérieur (application) le PIC 16F84 vous met à disposition 13 Entrées / Sorties programmables individuellement soit en entrée soit en sortie. Ces 13 E/S sont issues de 2 ports nommés PORT A pour les 5 E/S RA0 à RA4 et PORT B pour les 8 E/S RB0 à RB7.

La broche RA4 est multiplexée avec l'entrée horloge du Timer TMR0. Elle peut être utilisée soit:

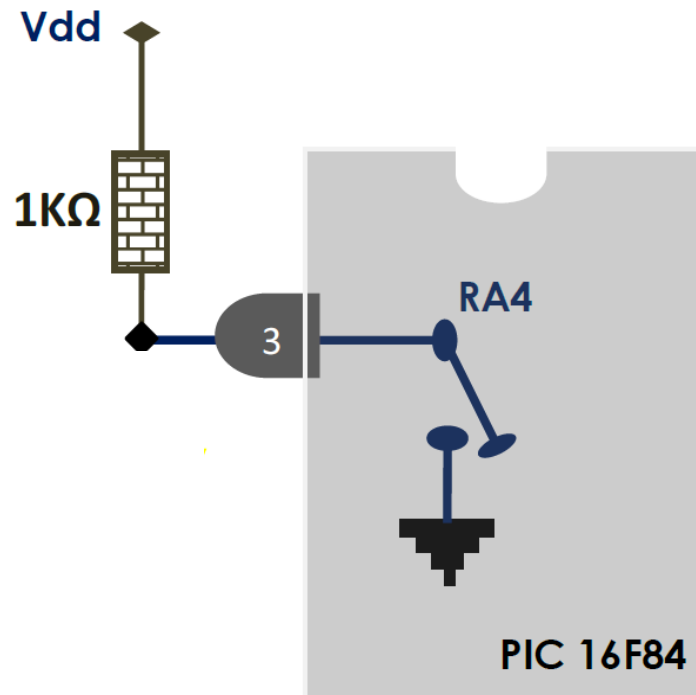
- ✓ Comme E/S normale du port A.
- ✓ Comme entrée horloge pour le Timer TMR0.



Les ports d'Entrées / Sorties

RA4 est une E/S à drain ouvert, si on veut l'utiliser comme sortie (pour allumer une LED par exemple), il ne faut pas oublier de mettre une résistance externe (de rappel) vers Vdd.

- ✓ Si RA4 est positionnée à 0, l'interrupteur est fermé, la sortie est reliée à la masse.
- ✓ Si RA4 est placée à 1, l'interrupteur est ouvert, la sortie est déconnectée d'où la nécessité de la résistance externe.



Le TIMER

Le PIC 16F84 dispose de deux TIMERS :

- ✓ Un à usage général (le TMR0) et
- ✓ Un autre utilisé pour le chien de garde (watch dog WDG).

a- Le TIMER TMR0

Le TIMER TMR0 permet au PIC 16F84 le contrôle (la gestion) du temps pour assurer le bon fonctionnement.

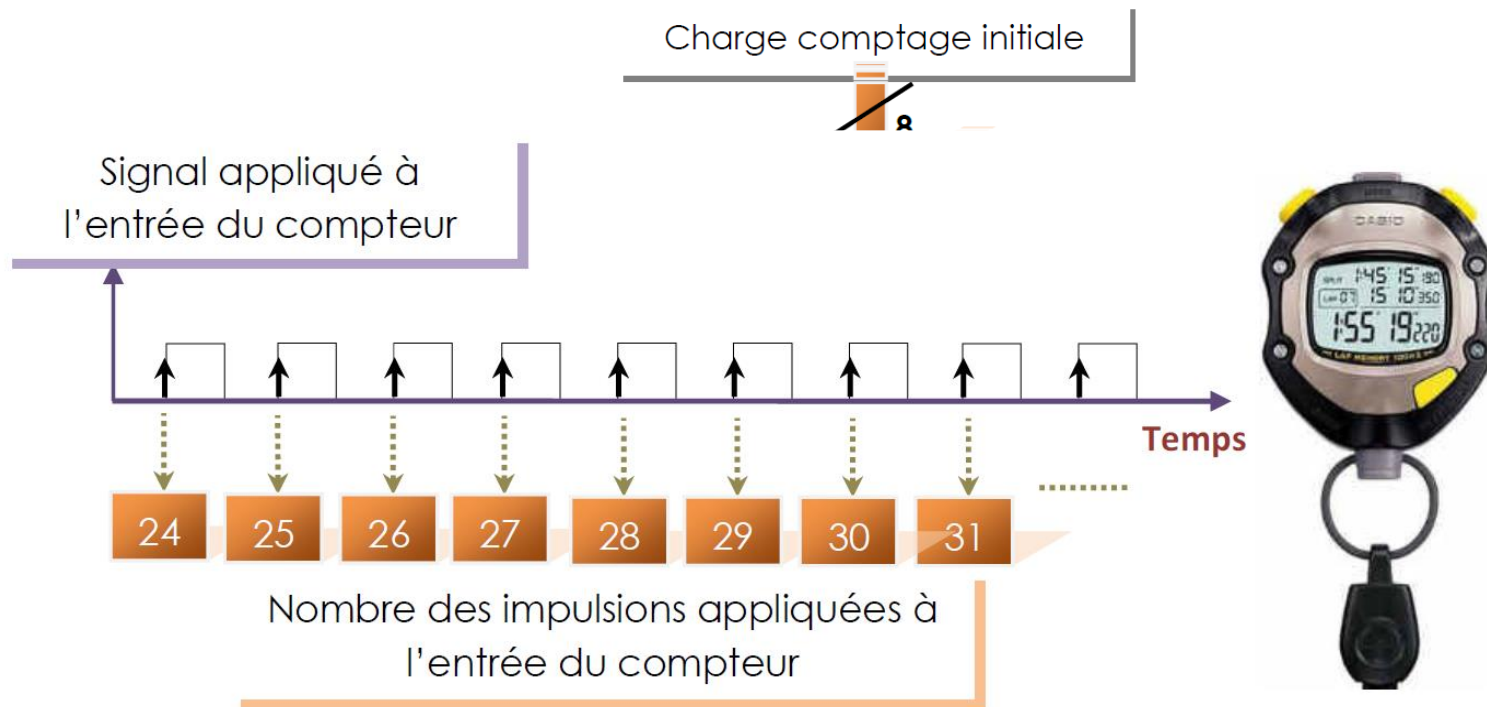
- ✓ Le TMR0 est un compteur 8 bits qui peut être chargé avec une valeur initiale quelconque.
- ✓ Il est ensuite incrémenté à chaque Top d'horloge (front montant (ou descendant)) du signal qui lui est appliqué jusqu'à ce que le débordement ait lieu (passage de FF à 00).

Le TIMER

a- Le TIMER TMR0

Donc, le TIMER TMR0 peut remplir deux fonctions:

- ✓ Temporisateur ou contrôle du temps. Son entrée d'incrémentation est alors l'horloge qui correspond au cycle instruction ($F_{osc}/4$).
- ✓ Compteur d'événements. Dans ce cas les d'impulsions d'entrées du TIMER sont fournies par la broche RA4/TOCK1.



Le TIMER

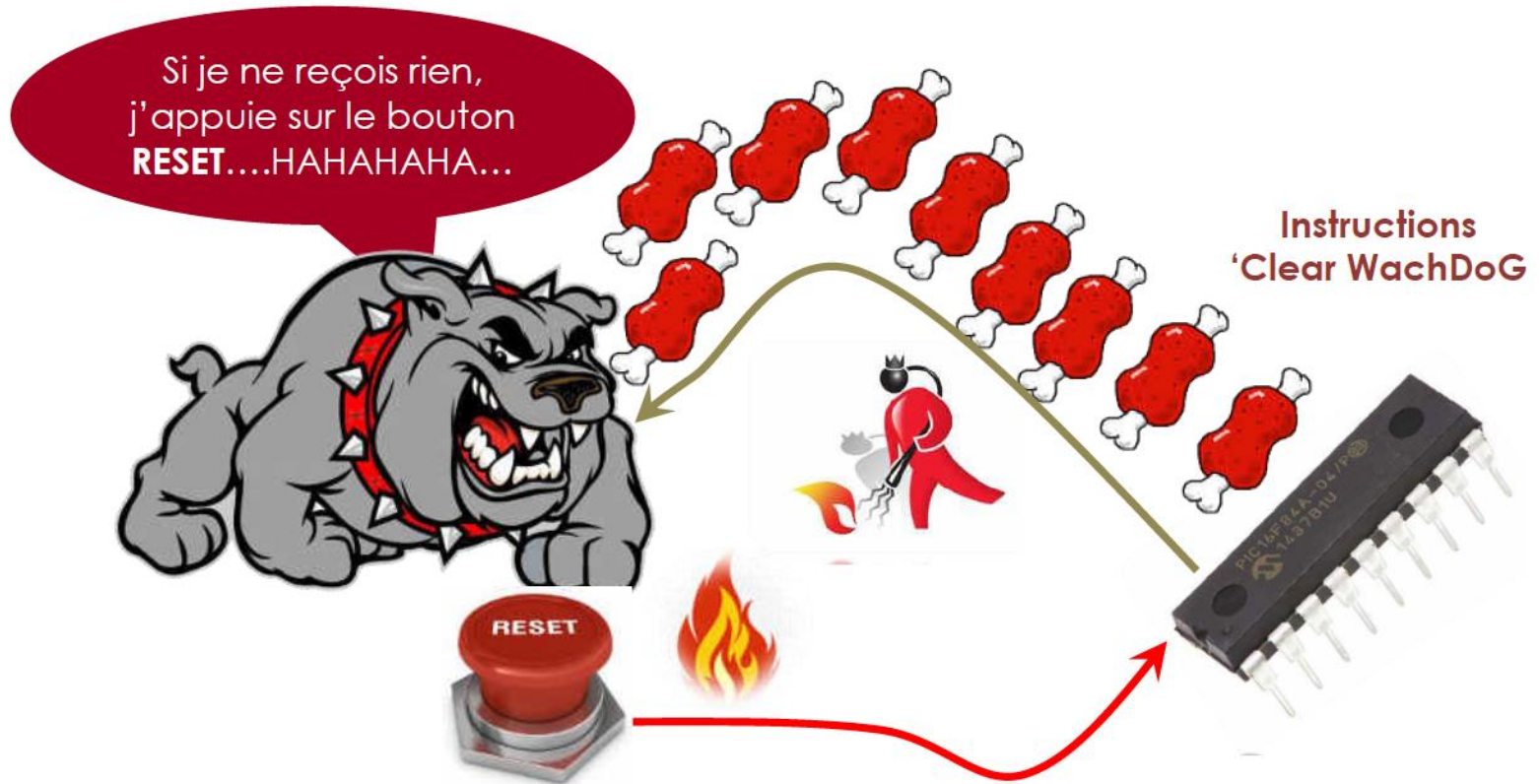
b- Le chien de garde (Watchdog TIMER)

Le WatchDog Timer (WDT) ou chien de garde est un mécanisme pour protéger le programme dans le PIC 16F84. Son rôle est de surveiller si le programme s'exécute toujours dans l'espace et dans le temps attribué, c'est-à-dire :

- ✓ Le programme ne s'est pas «égaré» dans une zone non valide du programme (parasite sur l'alimentation), ou
- ✓ S'il n'est pas bloqué dans une boucle infinie (bug du programme).

Le chien de garde (watchdog) est un dispositif matériel et logiciel qui permet de se prémunir contre les plantages accidentels. L'idée est de provoquer un RESET du CPU afin de relancer l'application (les données sont bien sur perdues). Le plantage est défini lorsque le programme n'est pas venu à temps faire signe au watchdog.

Le TIMER



Le TIMER

b- Le chien de garde (Watchdog TIMER)

- ✓ C'est un compteur 8 bits incrémenté en permanence (même si le PIC 16F84 est en mode SLEEP) par une horloge (RC) intégrée indépendante de l'horloge système.
- ✓ WatchDog Timer sert également à réveiller un PIC placé en mode SLEEP.
- ✓ Lorsqu'il déborde (WDT TimeOut), deux situations sont possibles :
 - ❖ Si le PIC 16F84 est en fonctionnement normal, le WDT TimeOut provoque un RESET. Ceci permet d'éviter de rester planté en cas de blocage du microcontrôleur par un processus indésirable non contrôlé.
 - ❖ Si le PIC 16F84 est en mode SLEEP, le WDT TimeOut provoque un WAKE-UP, l'exécution du programme continue normalement là où elle s'est arrêtée avant de rentrer en mode SLEEP. Cette situation est souvent exploitée pour réaliser des temporisations.

Le TIMER

c- Le mode SLEEP

Le PIC peut être placé en mode de consommation réduite à l'aide de l'instruction SLEEP. Dans ce mode, l'horloge système est arrêtée ce qui arrête l'exécution du programme.

Pour sortir du mode SLEEP, il faut provoquer un WAKE-UP, soit par programme (Soft) soit par un signal externe (Hard):

1. RESET externe (Hard) dû à l'initialisation du PIC en mettant l'entrée MCLR à 0 Niveau bas sur le pin MCLR ce qui provoque un reset à l'adresse 0000. Le PIC reprend l'exécution du programme à partir du début.
2. TimeOut du WDT (Soft) si celui-ci est validé. Le PIC reprend le programme à partir de l'instruction qui suit l'instruction SLEEP.
3. Interruption INT (sur RBO ou RBI (sur RB4-RB7)) (HARD) ou EEI (fin d'écriture dans l'EEPROM de données) (Soft).

Gestion des interruptions

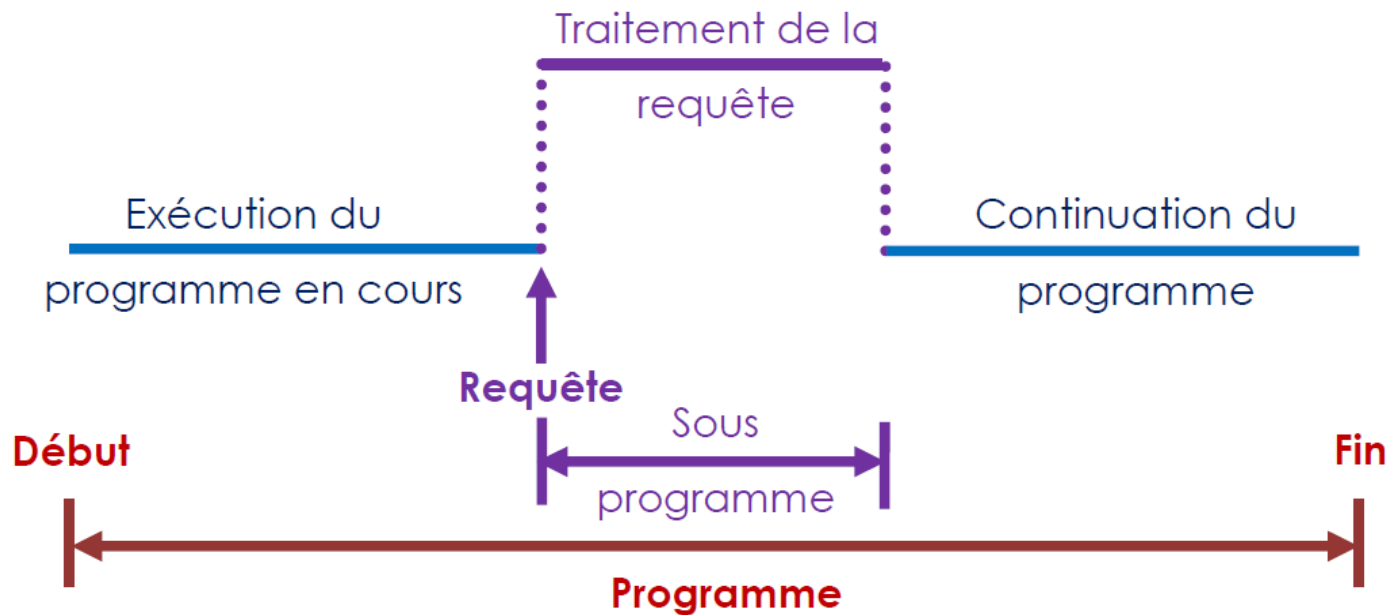
L'interruption est un mécanisme fondamental dans le fonctionnement des microcontrôleurs. Elle permet de prendre en compte des événements extérieurs et de leur associer un traitement particulier. Autrement dit, une interruption est un signal déclenché par un événement interne ou externe (envoyé au microcontrôleur par un périphérique ou un programme) pour l'avertir d'un événement à traiter (demande d'interruption ou requête d'interruption).

Une interruption (interne (Soft) ou externe (Hard)) provoque l'arrêt du programme principal pour aller exécuter une procédure d'interruption (`void interrupt()`). A la fin de cette procédure, le microcontrôleur reprend le programme à l'endroit où il s'était arrêté. La pile (stack) sert donc à sauvegarder les adresses de branchement vers les routines des interruptions.

☐ La séquence classique de fonctionnement d'une interruption est illustrée par le schéma de la figure ci-dessous.

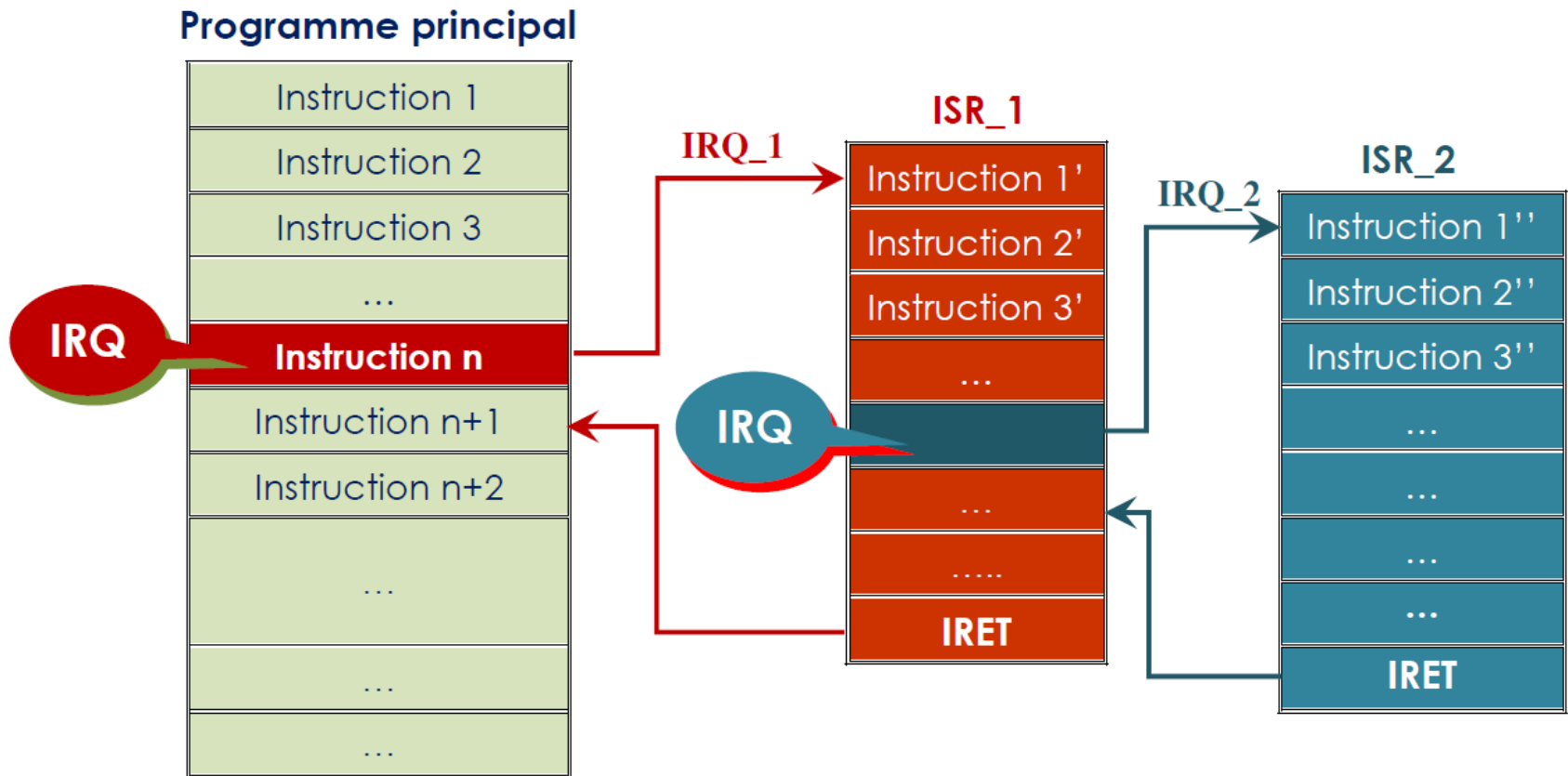
Gestion des interruptions

La séquence classique de fonctionnement d'une interruption est illustrée par le schéma de la figure suivante :



Gestion des interruptions

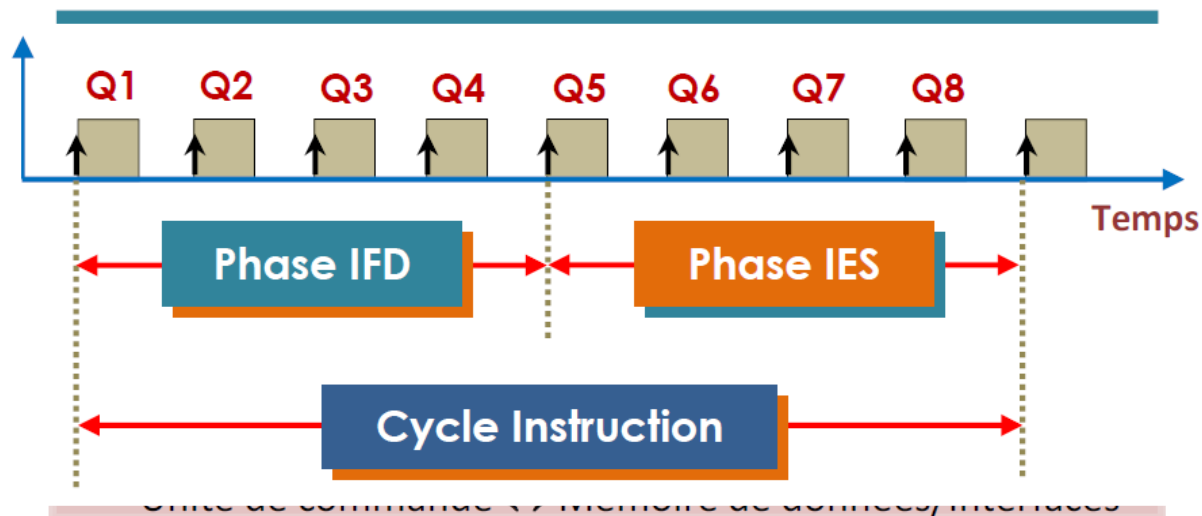
La séquence classique de fonctionnement d'une interruption est illustrée par le schéma de la figure suivante :



Principe de fonctionnement du PIC

La tâche d'un microcontrôleur est d'exécuter des programmes (Ensemble des instructions) répondant au cahier de charge de l'utilisateur. Dans le cas de l'architecture RISC, Une instruction est exécutée en deux phases, où chaque phase dure 4 cycles d'horloge :

1. Phase IFD = Phase de recherche et de décodage de l'instruction (Fetch and Decode (IFD));
2. Phase IES = Phase d'exécution effective de l'instruction et stockage le résultat (Execute and Save (IES));



Principe de fonctionnement du PIC

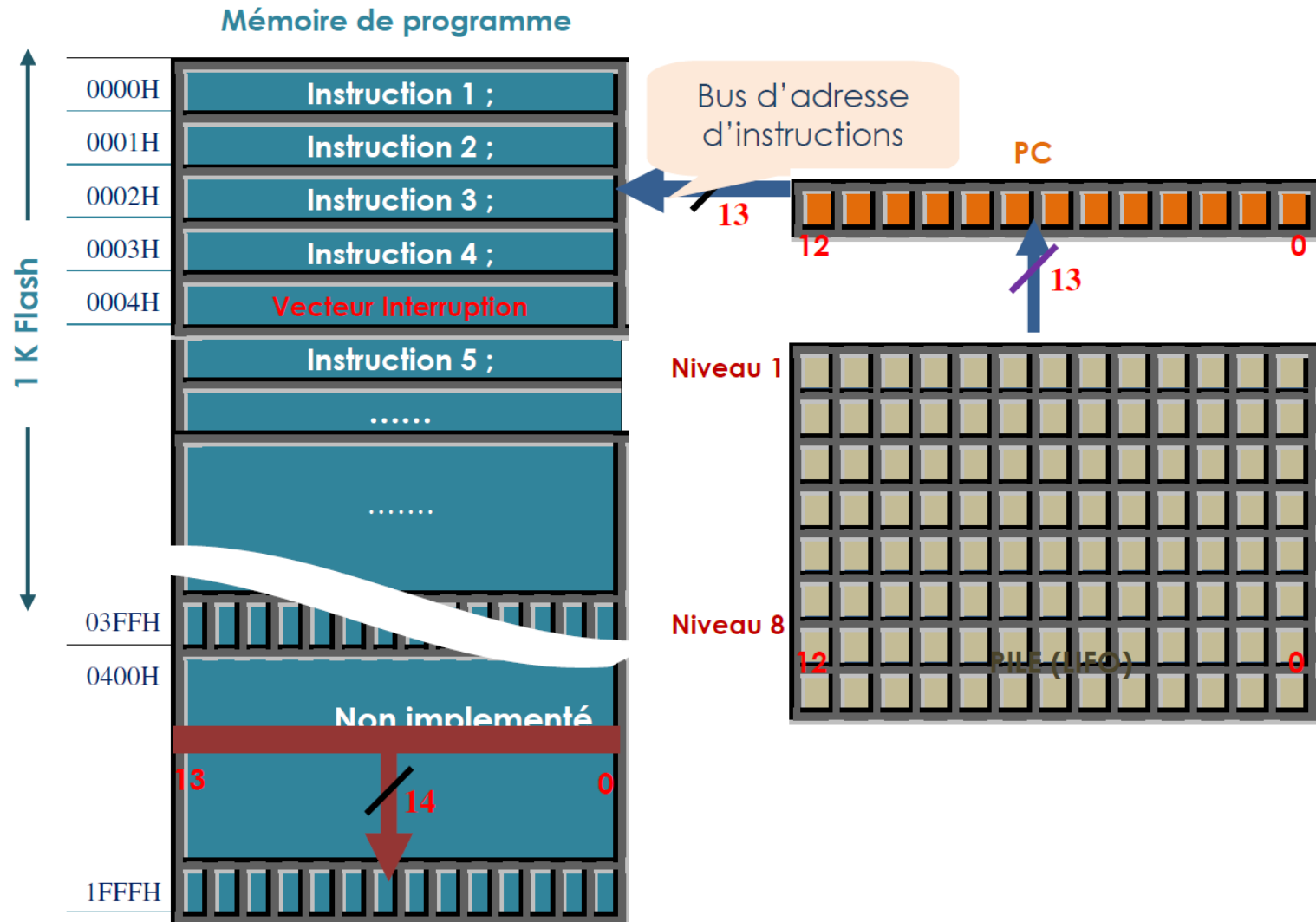
Déroulement d'un programme

Le déroulement d'un programme s'effectue d'une manière séquentielle. A la mise sous tension, le processeur va :

1. Chercher la première instruction qui se trouve à l'adresse 0000H de la mémoire de programme,
2. Exécuter cette première instruction.
3. Chercher la deuxième instruction à l'adresse 0001H et ainsi de suite.

Principe de fonctionnement du PIC

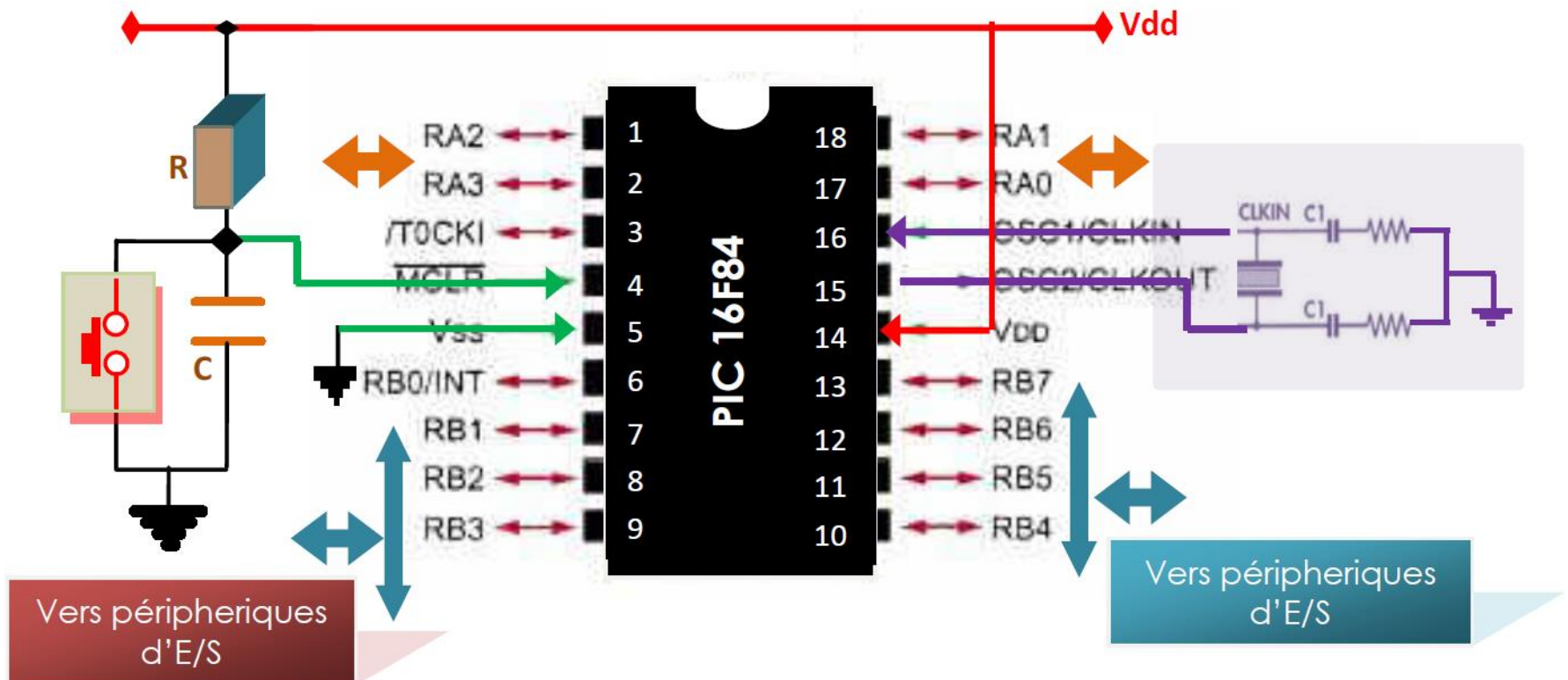
Déroulement d'un programme



Principe de fonctionnement du PIC

Mise en œuvre

L'utilisation et la mise en œuvre très simple des PICs les a rendus extrêmement populaire. Il est à noter que la société *MICROCHIP*, fabricant des PIC est devenu un leader mondial dans le domaine des microcontrôleurs devant d'autres concurrent à



Principe de fonctionnement du PIC

Mise en œuvre

Pour faire fonctionner le PIC, il suffit de:

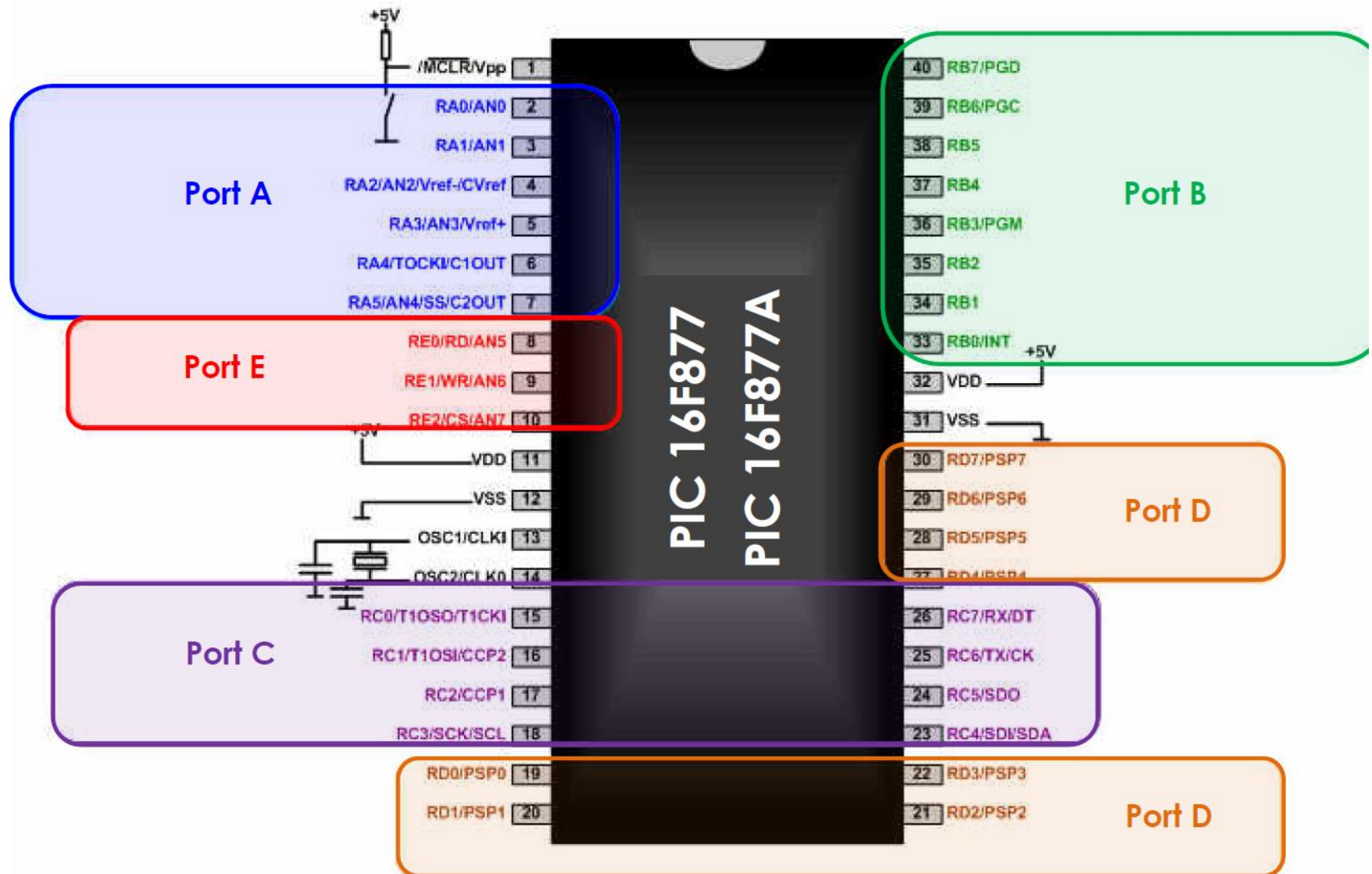
- ✓ D'alimenter le circuit par ses deux broches Vdd et Vss.
- ✓ De fixer la vitesse de fonctionnement voulue à l'aide d'un QUARTZ.
- ✓ De réaliser un système de réinitialisation (RESET) du microcontrôleur sans avoir à couper l'alimentation.
- ✓ D'écrire le programme en langage assembleur ou en C sur un ordinateur.
- ✓ De compiler pour le transformer en langage machine.
- ✓ De transférer le fichier résultant (programme de gestion de l'application développée) dans le PIC grâce à un programmeur.

Principe de fonctionnement du PIC

Les caractéristiques du PIC 16F84 fourni par Microchip :

- ✓ Mémoire de programme : 1KO, type Flash.
- ✓ Mémoire de données RAM : 68 octets.
- ✓ Mémoire de données E²PROM : 64 octets.
- ✓ Niveau de la pile : 8.
- ✓ Jeux d'instruction RISC : 35 de 14 bits.
- ✓ Fréquence max de travail : 10 MHz.
- ✓ Lignes E/S numérique : 13.
- ✓ Temporisateur : un pour l'utilisateur, un pour le Watchdog.
- ✓ Tension d'alimentation : 2 à 6 V continu.
- ✓ Boîtier : DIL 18 broches.

Un autre exemple de Microcontrôleur : le PIC 16F877



Architecture logicielle du PIC 16F84

Les processus de développement d'un programme à exécuter en assembleur sur un PIC sont :

1. Ecrire un programme en langage assembleur du PIC dans un fichier texte et le sauvegarder avec l'extension .asm.
2. Compiler ce programme avec l'assembleur MPASM fourni par Microchip. Le résultat est un fichier exécutable avec l'extension .hex contenant une suite d'instruction compréhensible par le PIC.
3. Transplanter le fichier .hex dans la mémoire programme du PIC (mémoire FLASH) à l'aide d'un programmeur adéquat.
4. Mettre le PIC dans son montage final puis mettre sous tension.



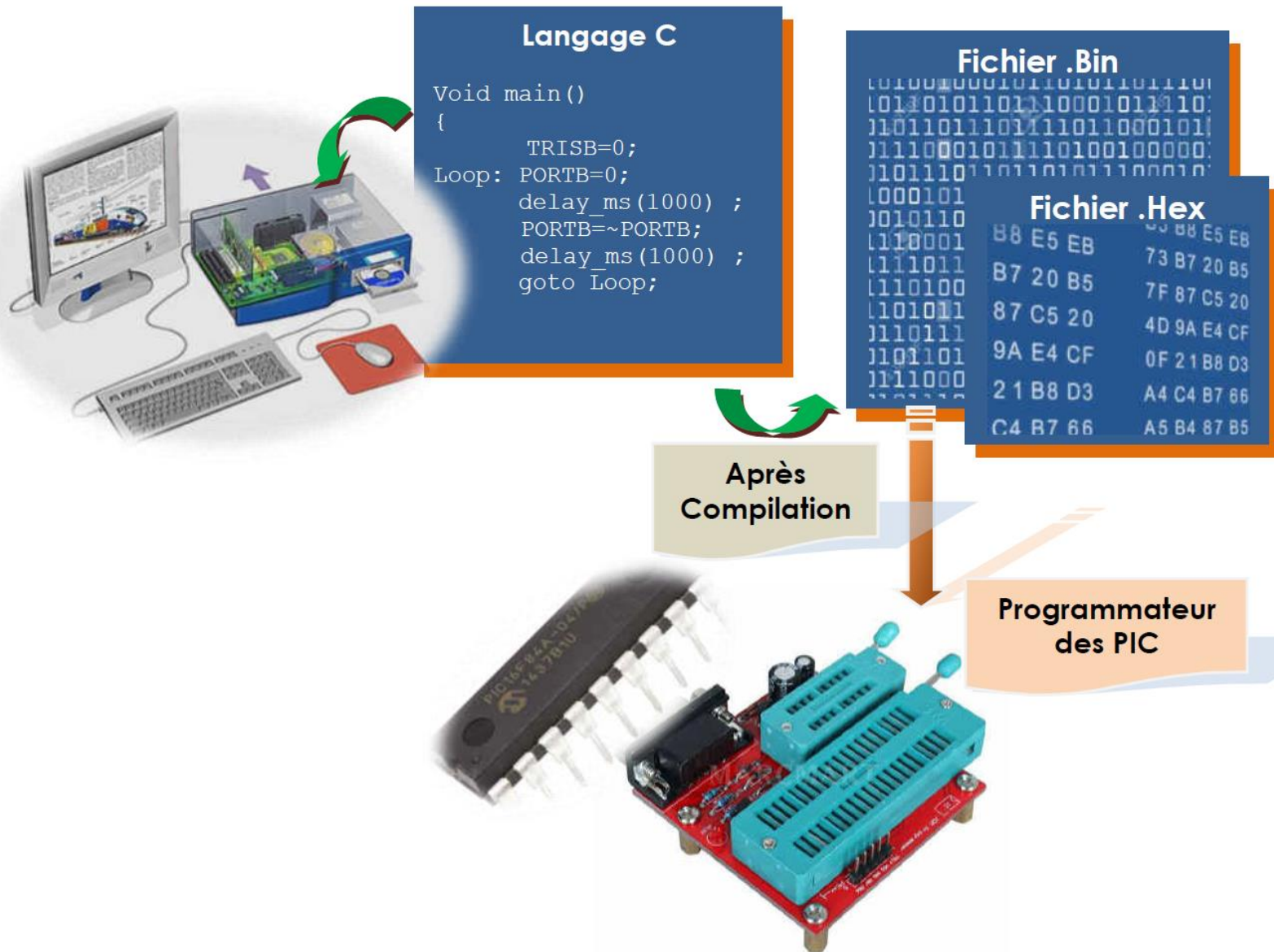
Structure d'un programme

Le microcontrôleur exécute le programme chargé dans sa mémoire FLASH. Les mots binaires sur 14 bits pour le cas de PIC 16F84 sont considérés par le CPU comme le système de gestion (commande) de la carte développée.

Pour obtenir le fichier de 'commande' binaire (ou même Hexadécimal), il est nécessaire (voir obligatoire) d'utiliser des langages de programmation (compilateurs) qui utilisent des abréviations (code plus lisible pour le programmeur), à savoir :

- ✓ Un langage bas niveau comme l'assembleur ou
- ✓ Des langages évolués comme le langage C.

Structure d'un programme



Structure d'un programme

- ✓ Il est à noter que les microcontrôleurs ne travaillent qu'avec des langages de bas niveau (Assembleur). Cependant, ces langages étant difficiles à comprendre et à appréhender pour les êtres humains, c'est la raison pour laquelle les fabricants des microcontrôleurs ont prévu de les programmer à l'aide des langages de plus haut niveau (C, C++, Python, etc.)
- ✓ Chaque type des microcontrôleurs et microprocesseurs possède son propre assembleur ou son propre jeu d'instruction.
- ✓ Une fois le code écrit dans le langage associé au type de microcontrôleur, il existe des programmes qui permettent de transformer des langages de haut niveau (Python, C, C++, etc.) vers ces langages machines. Ce sont les compilateurs.
- ✓ Dans le commerce, il y'a plusieurs variétés de compilateurs des différents fabricants et avec différents langages de haut niveau.
- ✓ Dans le cas général, ces compilateurs sont intégrés dans un environnement de développement propre au type de microcontrôleurs (IDE) et ils sont associés à une série de bibliothèques de fonctions permettant de gérer les traitements internes ou/et de communiquer avec des périphériques d'Entrées/Sorties.

Programmer avec le PIC 16F84

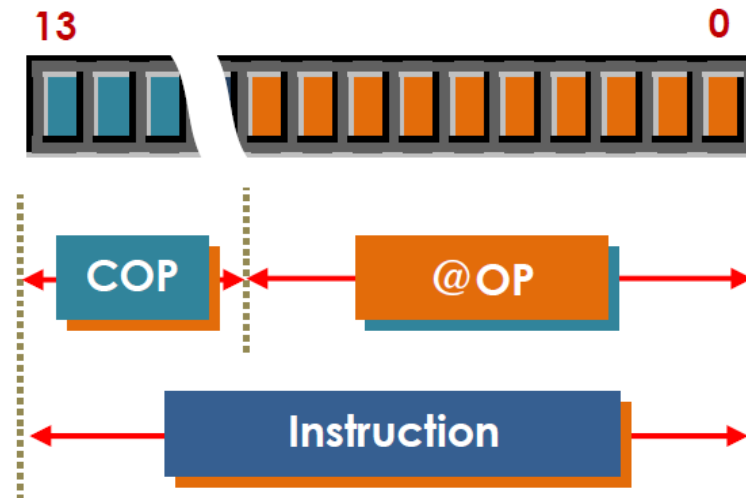
Certains registres sont indispensables au déroulement des programmes écrits en assembleur du PIC 16F84.

1. Le registre de travail W (Work), associé directement à l'UAL indépendant des registres spécifiques (SFR) où le processeur ne peut déplacer une donnée ou effectuer une opération arithmétique ou logique sur un registre (SFR ou GPR) qu'en utilisant le registre de travail W.
2. Le registre d'état STATUS : Son rôle est de donner des indications liées à l'exécution de la dernière instruction par le CPU (Etat du processeur). Il permet aussi de sélectionner les différentes pages mémoires en RAM ("banques").
3. Le compteur programme PC: Il pointe sur la prochaine instruction à exécuter. Il fait partie de SFR.
4. Lors de la mise sous tension, tous les registres spécifiques (SFR) sont placés dans un état déterminé.

Les instructions du PIC 16F84

Tous les PICs Mid-Range ont un jeu de 35 instructions. Chaque instruction est codée sur un mot de 14 bits qui contient le code opération (COP) ainsi que l'opérande (@OP).

- ✓ COP : Permet de spécifier le type de l'opération à effectuer.
- ✓ @OP : Permet de spécifier le ou les arguments de l'opération (un ou plusieurs opérande).



Les instructions du PIC 16F84

Chaque ligne du programme assembleur peut contenir jusqu'à 4 types d'informations :

- ✓ Une étiquette (adresse de l'instruction en mémoire).
- ✓ L'instruction en code mnémonique.
- ✓ Le ou les opérandes.
- ✓ Des commentaires.

Etiquette :	Code Opération (COP)	Opérande (s) (@OP)	Commentaires ;
-------------	-------------------------	-----------------------	----------------

- Une ligne contient 255 caractères maximum.
- Si on utilise par exemple un quartz de 4MHz (Sachant que l'horloge fournie au PIC est prédivisée par 4), on obtient donc 1 000 000 cycles/seconde = 1MIPS (1 Million d' Instructions Par Seconde).
- Le jeu d'instructions du PIC 16F84 contient :
 - ❖ 9 opérations sur des octets.
 - ❖ 10 opérations sur des bits.
 - ❖ 11 opérations de contrôle

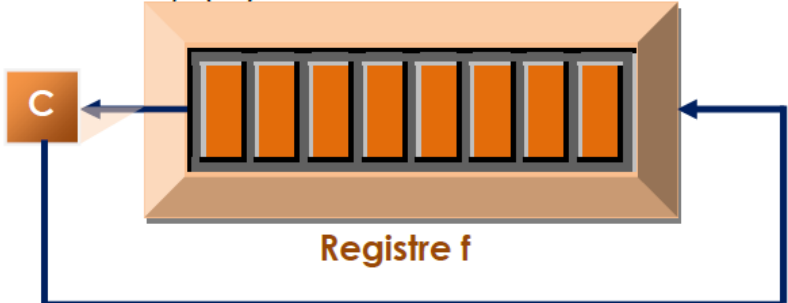
Les instructions du PIC 16F84

Code mnémonique	Description
Instructions de Transfert des données	
movf f, d ;	Si d=0, Copie le contenu de f dans W, sinon copie f dans f ;
movlw k ;	Chargement (littéral) d'une valeur k dans W ;
movwf f ;	Copier le contenu de W dans le registre f ;
clrf f ;	Placer zéro dans f ;
clrw	Placer zéro dans W (passage du bit Z dans STATUT '1' ;
Clrwdt ;	Initialiser (Remise à zéro) le timer du chien de garde ;
swapf f,d ;	Permuter (échanger) les deux quartets de f. Résultat dans W si d=0 ou dans f si d=1 ;

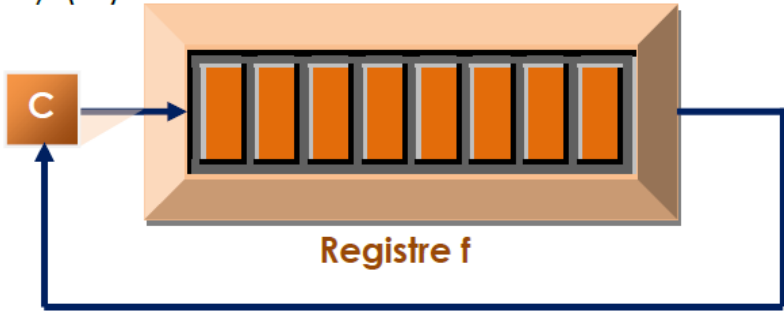
Les instructions du PIC 16F84

Instructions arithmétiques	
addlw k ;	Ajoute une constante k à W. Résultat dans W ;
addwf f, d ;	Ajouter W à f. Résultat dans W si d=0 ou dans f si d=1 ;
btfsc f, b ;	Tester le bit b de f. Incrémente PC si b=0 (Sauter la prochaine instruction) ;
btfss f, b ;	Tester le bit b de f. Incrémente PC si b=1 (Sauter la prochaine instruction) ;
incf f, d ;	Incrémente f ;
incfsz f, d ;	Incrémenter f. Résultat dans W si d=0 ou dans f si d=1 ;
sublw k ;	Sustrait W d'une constante k ($W \leftarrow k - W$) ;
subwf f, d ;	Soustrait W de f. Résultat dans W si d=0 ou dans f si d=1 ;
decf f, d ;	Décrémenter f. Résultat dans W si d=0 ou dans f si d=1 ;
decfsz f, d ;	Décrémenter f. Si f=0, incrémente PC (Sauter la prochaine instruction). Résultat dans W ou dans f ;

Les instructions du PIC 16F84

Instructions logiques	
andlw k ;	Effectuer un ET logique entre une la constante k et W. Résultat dans W ;
andwf f, d ;	Effectuer un ET logique entre W et f. Résultat dans W si d=0 ou dans f si d=1 ;
iorlw k ;	Effectuee un OU logique entre une constante et W ;
iorwf f, d ;	Effectuer un OU logique entre W et f. Résultat dans W si d=0 ou dans f si d=1 ;
comf f, d ;	Complémenter f à 1. Résultat dans W si d=0 ou dans f si d=1 ;
rlf f, d ;	Effectuer une rotation de bits à gauche à travers Carry (C) ;  Register f

Les instructions du PIC 16F84

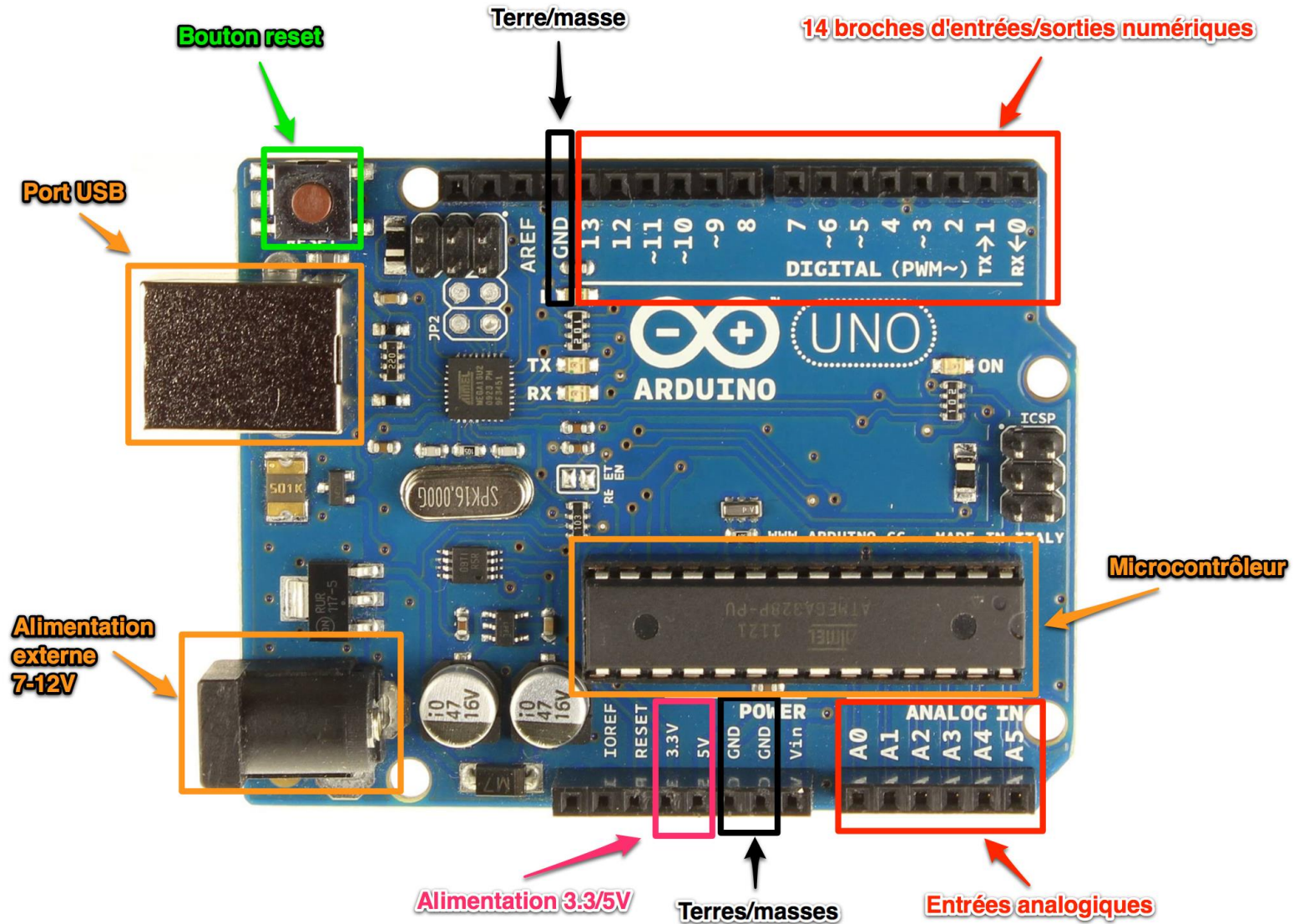
rrf f, d ;	Effectuer une rotation de bits à droite à travers Carry (C) ; 
xorlw k ;	Effectue un OU exclusif entre une constante et W. Resultat dans W ;
xorwf f, d ;	OU exclusif entre W et f. Résultat dans W si d=0 ou dans f si d=1 ;
bcf f, b ;	Remise à zéro le bit b de f ;
bsf f, b ;	Mise à 1 le bit de f à 1 ;

Les instructions du PIC 16F84

Instructions de branchement (saut) / mise en veille	
goto Label ;	Branchement à une adresse Label (Etiquette) $PC \leftarrow @SAUT = \text{Label} ;$
sleep ;	Mise en mode veille le PIC ;
nop	Aucune opération n'aura lieu
call Label ;	Empile PC et affecte PC de l'adresse d'un sous programme (Etiquette): $PILE \leftarrow PC+1 ; PC \leftarrow @SAUT = \text{Label (Etiquette)} ;$
retfie ;	Retour d'interruption ;
retlw k ;	Retour de sous programme avec chargement de valeur k dans W.
return ;	Retour de sous programme

A collage of various electronic circuit boards, including Arduino Uno, Raspberry Pi, and other microcontroller boards, illustrating the field of embedded systems. The boards are of different colors (blue, red, black, green) and shapes, featuring various components like chips, capacitors, and connectors. Some boards have labels like 'Arduino', 'Raspberry Pi', and 'WiringPi'. The boards are arranged in a scattered, overlapping manner on a white background.

La plateforme Arduino

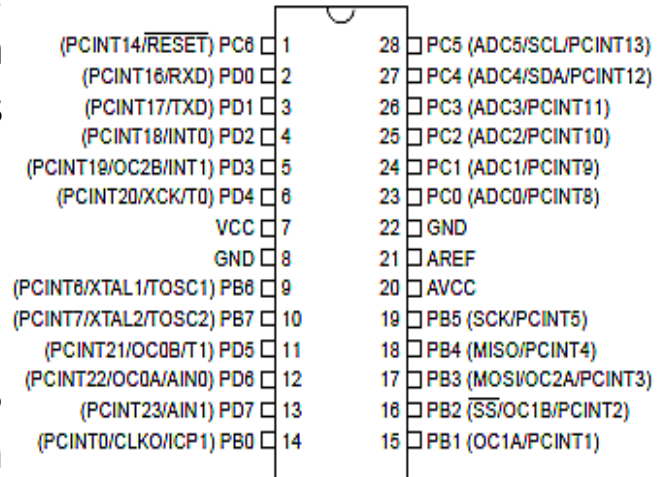


La plateforme Arduino

Le microcontrôleur ATMEL - ATmega328 :

Le microcontrôleur de la carte Arduino UNO est le ATmega328. C'est un microcontrôleur ATMEL de la famille AVR 8bits. Les principales caractéristiques sont:

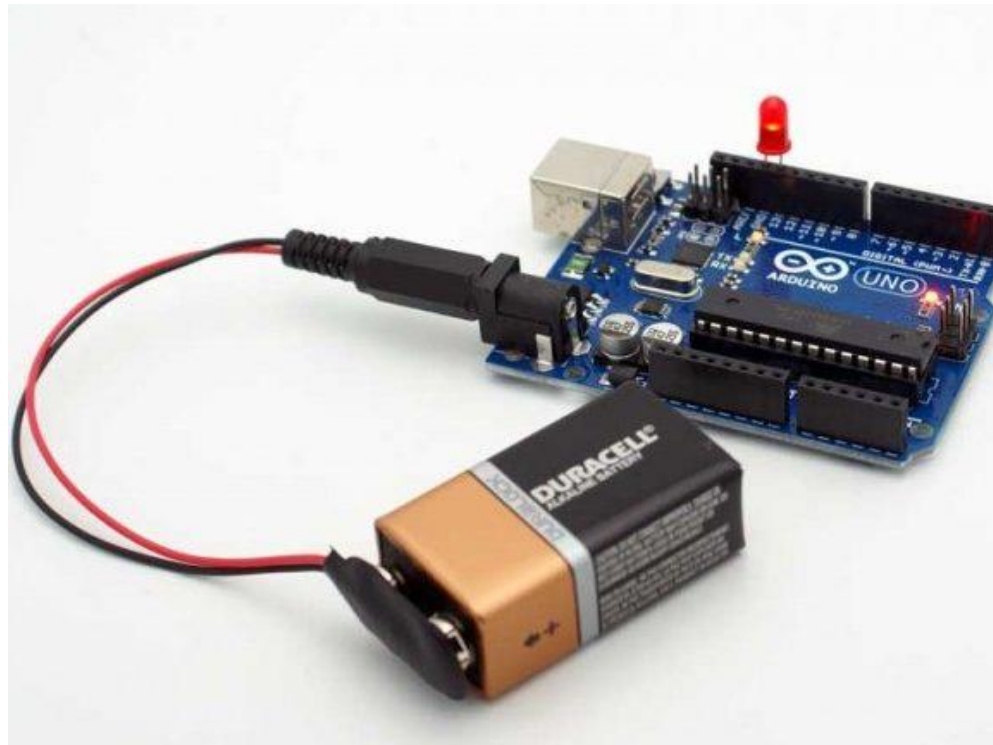
- ✓ FLASH = mémoire programme de 32Ko
- ✓ SRAM = données (volatiles) 2Ko
- ✓ EEPROM = données (non volatiles) 1Ko
- ✓ Digital I/O (entrées-sorties Tout Ou Rien) = 3 ports PortB, PortC, PortD (soit 23 broches en tout I/O)
- ✓ Timers/Counters : Timer0 et Timer2 (comptage 8 bits), Timer1 (comptage 16bits). Chaque timer peut être utilisé pour générer deux signaux PWM.



La plateforme Arduino

L'alimentation :

Pour fonctionner, une carte Arduino a besoin d'une alimentation. Le microcontrôleur fonctionnant sous 5V, la carte peut être alimentée en 5V par le port USB ou bien par une alimentation externe qui est comprise entre 7V et 12V. Un régulateur se charge ensuite de réduire la tension à 5V pour le bon fonctionnement de la carte.

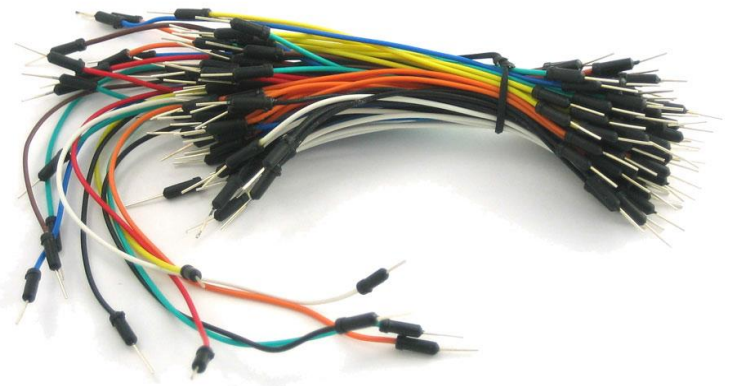
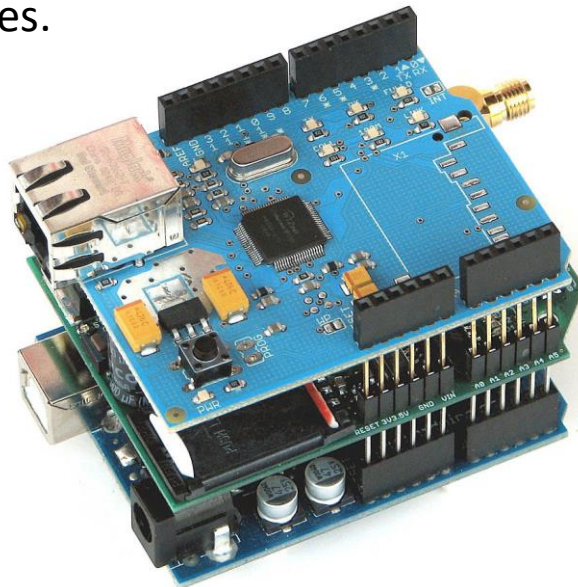


La plateforme Arduino

La connectique :

A part une LED sur la broche 13, la carte Arduino ne possède pas de composants (résistances, diodes, moteurs...) qui peuvent être utilisés pour un programme. Il est nécessaire de les rajouter. Mais pour cela, il faut les connecter à la carte. C'est là qu'interviennent les connecteurs, aussi appelés broches (pins, en anglais). Sur les Arduino et sur beaucoup de cartes compatibles Arduino, les broches se trouvent au même endroit. Cela permet de fixer des cartes d'extension, appelée shields en les empilant.

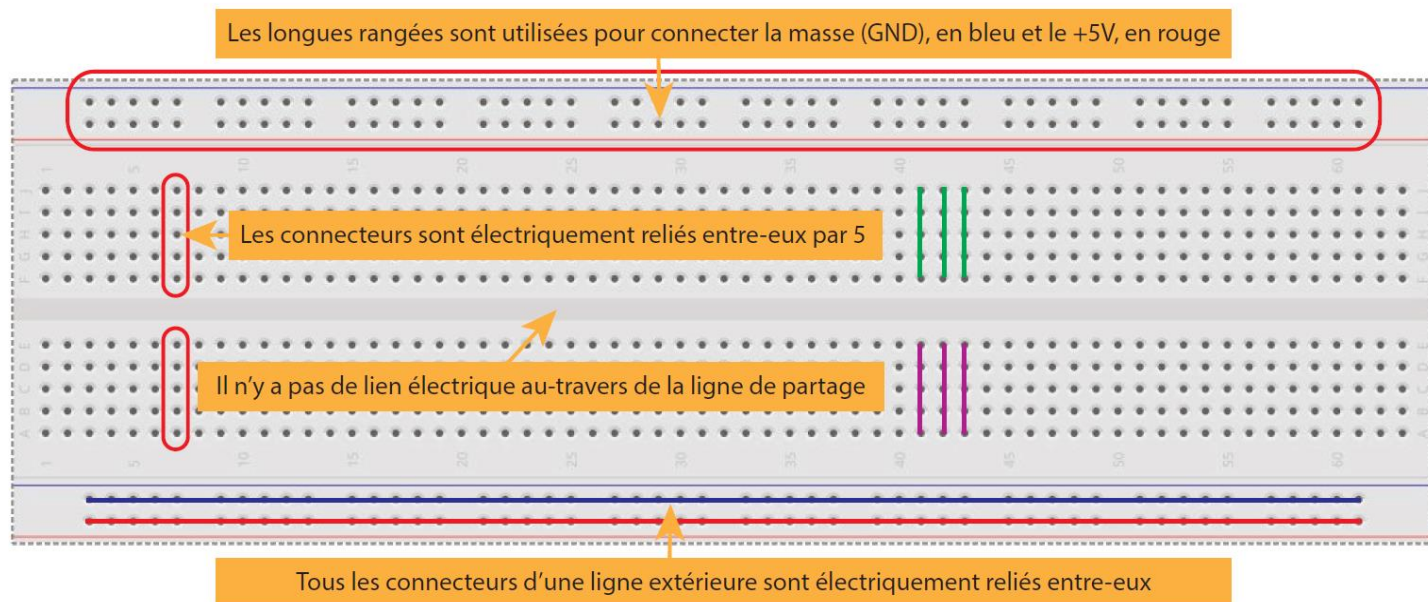
Les connexions entre les composants sont réalisées par des jumpers, sortes de petits câbles.



La plateforme Arduino

La platine d'expérimentation :

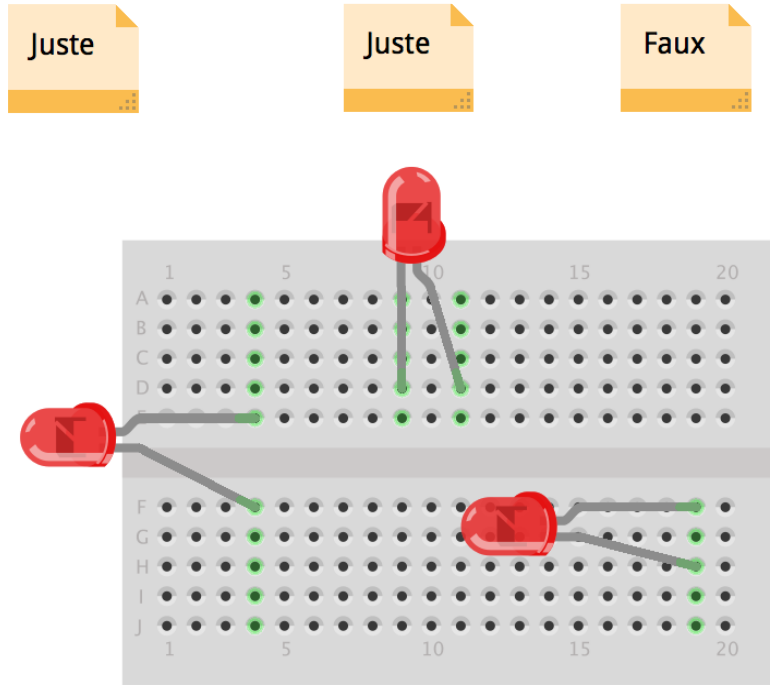
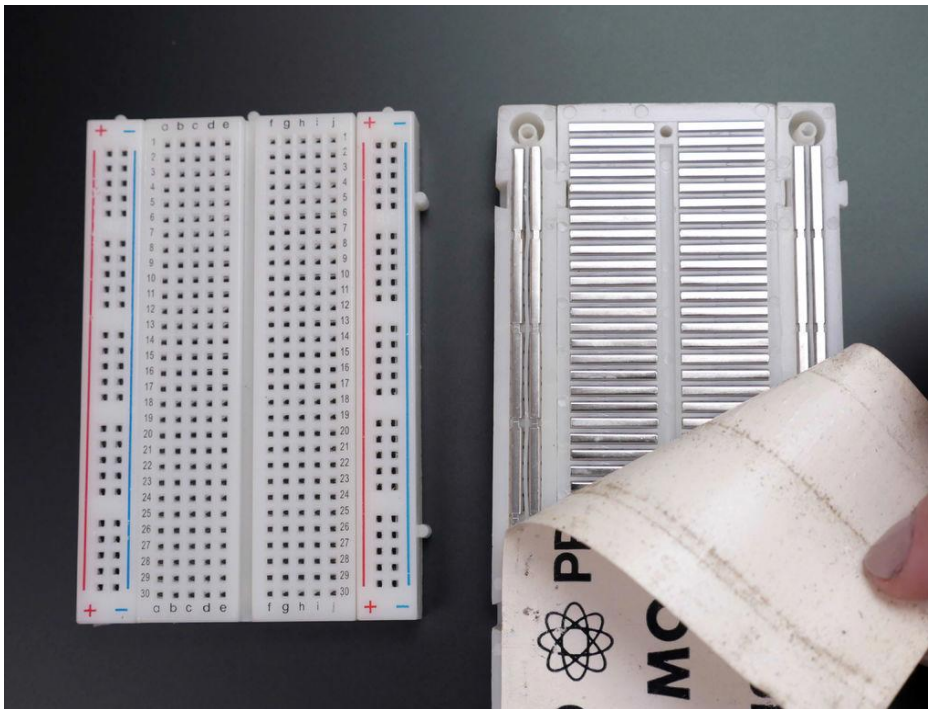
Une platine d'expérimentation (appelée breadboard) permet de réaliser des prototypes de montages électroniques sans soudure et donc de pouvoir réutiliser les composants.



La plateforme Arduino

La platine d'expérimentation :

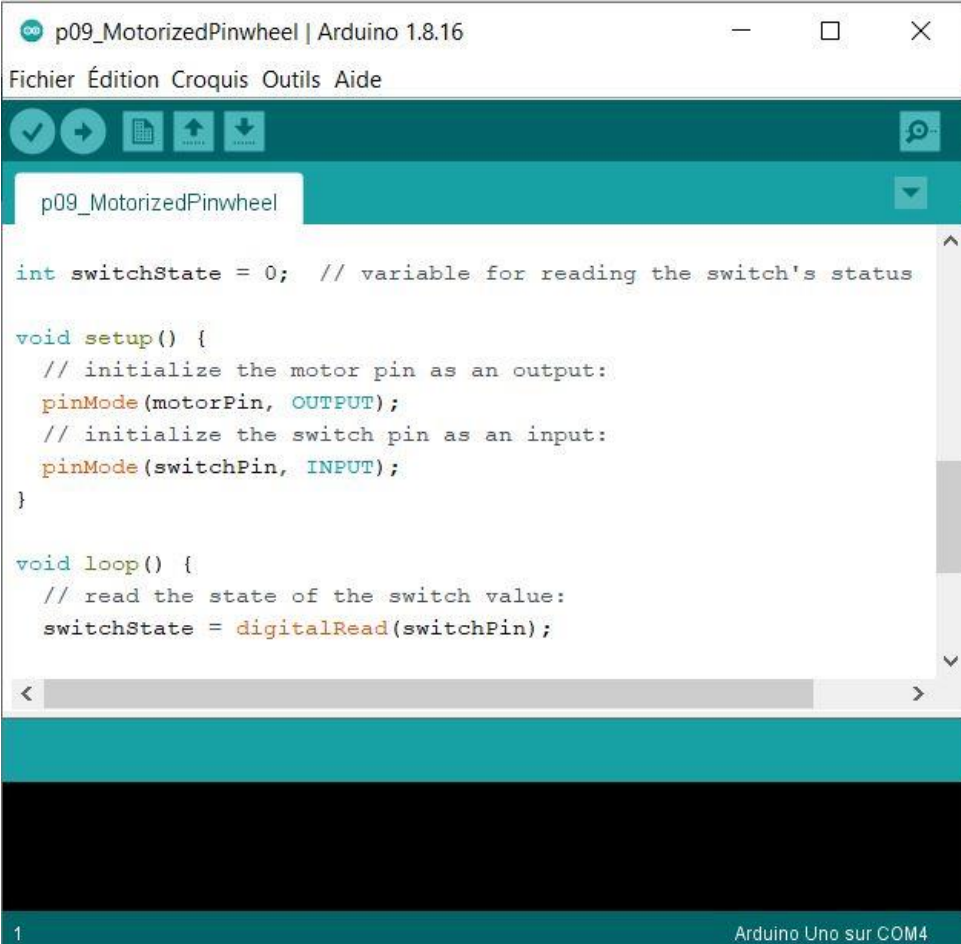
Une platine d'expérimentation (appelée breadboard) permet de réaliser des prototypes de montages électroniques sans soudure et donc de pouvoir réutiliser les composants.



La plateforme Arduino

Le logiciel Arduino IDE :

L'environnement de développement (IDE) d'Arduino fonctionne sur Mac, Windows et Linux. C'est grâce à ce logiciel que nous allons créer, tester et envoyer les programmes sur l'Arduino. Les opérations de compilation et de chargement dans la mémoire du microcontrôleur étant ramenées à des clicks sur des boutons dans l'IDE (très simple). La communication entre le PC et la carte se fait via le port USB, moyennant installation d'un driver adapté sur le PC (fourni par Arduino)



The screenshot shows the Arduino IDE window titled "p09_MotorizedPinwheel | Arduino 1.8.16". The menu bar includes "Fichier", "Édition", "Croquis", "Outils", and "Aide". The toolbar contains icons for opening, saving, and uploading. The file explorer on the left shows "p09_MotorizedPinwheel". The main editor displays the following code:

```
int switchState = 0; // variable for reading the switch's status

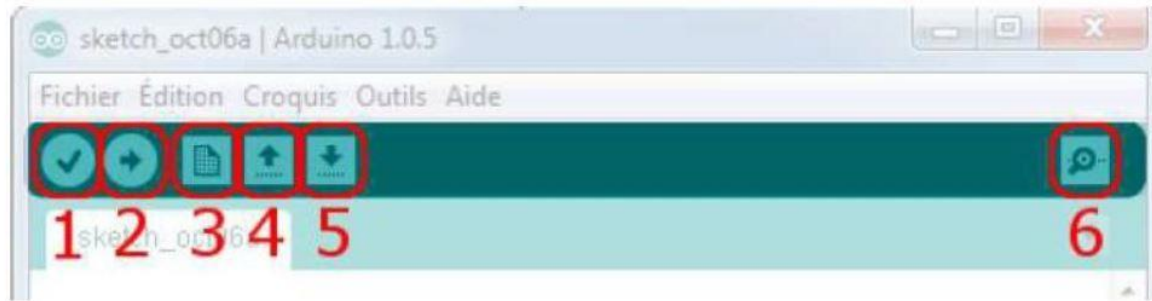
void setup() {
  // initialize the motor pin as an output:
  pinMode(motorPin, OUTPUT);
  // initialize the switch pin as an input:
  pinMode(switchPin, INPUT);
}

void loop() {
  // read the state of the switch value:
  switchState = digitalRead(switchPin);
}
```

The status bar at the bottom indicates "1" and "Arduino Uno sur COM4".

La plateforme Arduino

Le logiciel Arduino IDE :



1		Verify : Ce bouton permet de vérifier le programme, il actionne un module qui cherche les erreurs dans votre programme
2		Upload : Charge (téléverse) le programme dans la carte Arduino
3		New : pour créer un nouveau programme (sketch)
4		Open : pour ouvrir un programme existant
5		Save : pour sauvegarder le programme
6		Verify : pour visualiser les données transmises par le port série

La plateforme Arduino

Le logiciel Arduino IDE :

L'IDE Arduino est basé sur le langage C. Le C est un langage incontournable qui en a inspiré beaucoup d'autres. Inventé dans les années 70, il est toujours d'actualité dans la programmation système et la robotique.

Syntaxe du langage (Arduino) :

La syntaxe d'un langage de programmation est l'ensemble des règles d'écritures liées à ce langage. La structure de base du langage de programmation Arduino est assez simple et comprend au moins deux parties (**Setup()** et **loop()**). Ces deux parties, ou fonctions, contiennent des blocs d'instructions et sont impérativement requises pour que le programme fonctionne.

La fonction setup() doit suivre la déclaration des variables au tout début du programme. Il s'agit de la première fonction à exécuter dans le programme. Elle est exécutée une seule fois et sert à établir le mode d'une broche (pinMode) ou à initialiser la communication série.

La plateforme Arduino

Syntaxe du langage (Arduino) :

La fonction ***setup()*** doit suivre la déclaration des variables au tout début du programme. Il s'agit de la première fonction à exécuter dans le programme. Elle est exécutée une seule fois et sert à établir le mode d'une broche (pinMode) ou à initialiser la communication série.

La fonction ***loop()*** suit immédiatement et comprend le code à exécuter en continue – lisant les capteurs en entrée et déclenchant les actionneurs en sortie, etc. Cette fonction est le noyau de tout programme Arduino et réalise l'essentiel du travail.

```
void setup()  
{  
    Blocs instructions ;  
}
```

```
void loop()  
{  
    Blocs instructions ;  
}
```

La plateforme Arduino

Syntaxe du langage (Arduino) :

Les fonctions void setup() et void loop() sont les 2 fonctions obligatoires en arduino, mais d'autres peuvent être créées

Les accolades {} : Les accolades (ouvrantes ou fermantes) définissent le début et la fin d'une fonction ou d'un bloc d'instructions, comme pour la fonction void loop() ou les instructions if et for. L'environnement Arduino inclue un système de vérification des accolades. Il suffit de cliquer sur une accolade pour voir sa « paire » se surligner.

Le point-virgule : Le point-virgule est utilisé pour finir une instruction et séparer les éléments d'un programme. On l'utilise, par exemple, pour séparer les éléments de la boucle for.

Les commentaires // ou /*.....*/ : Un commentaire (sur une ligne // ou sur un paragraphe /*.....*/) est une zone de texte qui est ignoré par le programme lors de la compilation et qui est utilisé afin de décrire le code ou pour le commenter pour faciliter sa compréhension auprès de personnes tiers.

La plateforme Arduino

Les variables : Une variable est une façon de nommer et de stocker une information/valeur afin de l'utiliser plus tard dans le programme. Comme son nom l'indique, une variable est une valeur qui peut être continuellement modifiée à l'opposé d'une constante qui ne change jamais de valeur. Une variable a besoin d'être déclarée et on peut lui assigner une valeur que l'on veut stocker, mais cela est optionnel.

Une variable peut être déclarée à de nombreux endroits dans le programme, l'emplacement de sa déclaration détermine quelles parties du programme auront accès à cette variable.

```
int v_1=0; // déclare v_1 et lui assigne la valeur 0
int v_2=analogRead(2); /* assigne la valeur de la broche
                        analogique 2 à la variable v_2*/
```

La plateforme Arduino

Les types de données:

byte : Byte permet de stocker une valeur entière sur 8 bits (pas de chiffre à virgule donc). Cela permet d'aller de 0 à 255.

int : Int (pour integers, entier) est le type de base pour les chiffres (sans virgule) des variables. Le stockage se fait sur 16 bits, permettant d'aller de 32 767 à – 32 768.

long : Il s'agit d'un type de variable de grande taille : la donnée (chiffre sans virgule) est stockée sur 32 bits. Cela permet d'aller de 2 147 483 647 à – 2 147 483 648.

float : Ce type permet de stocker les chiffres « flottant » ou chiffre à virgule. Le float bénéficie d'une meilleure résolution : ils sont stockés sur 32 bits.

bool : Le type bool prend 2 valeurs : Vrai (true ou 1) ou Faux (false ou 0). Il est stocké sur 8 bits.

La plateforme Arduino

Les types de données:

char : Le type char est utilisé pour stocker une lettre. Le stockage se fait sur 8 bits et permet de stocker une seule Note : Pour assigner une valeur à la variable char, on l'écrit entre ''.

String : Il ne s'agit pas exactement d'un type de variable, il s'agit d'une classe. La classe String est comparable à une variable possédant ses propres fonctions. String permet de stocker une chaîne de caractères, comme pourrait le faire un tableau de caractère. String permet de réaliser des opérations sur les chaînes de caractères plus complexes que ce qui se ferait sur un tableau de caractères.

La plateforme Arduino

Les opérations arithmétiques :

Les opérations d'arithmétique incluent les additions, les soustractions, les multiplications et les divisions. Elles retournent respectivement la somme, la différence, le produit et le quotient des 2 opérands. L'opération est réalisée en utilisant les types de données des opérands, de façon à ce que, par exemple, 9 / 2 donnera 4 plutôt que 4,5 car 9 et 2 sont des entiers (int), or le type « int » ne peut être un chiffre à virgule. Cela veut aussi dire que le résultat peut être trop grand pour le type de variable initiale.

Les opérations d'incrémentement :

Les opérateurs d'incrémentement permettent de réaliser une opération simple, ils sont communément utilisés dans la boucle for. Il en existe 2 : l'incrémentement (++) et la décrémentation (--).

`x++` // est équivalent à `x=x+1` (incrémentement de 1)

`x--` // est équivalent à `x=x-1` (décrémentement de 1)

La plateforme Arduino

Les opérations de comparaison :

La comparaison d'une variable ou d'une constante avec une autre est souvent utilisée afin de tester si une condition spécifique est vraie ou non. Dans les exemples suivants, l'opérateur est utilisé pour indiquer une condition :

```
x==y // retourne vrai si x est égale à y
x!=y // x est différent de y
x>y // x supérieur à y
x>=y // x supérieur ou égale à y
x<y // x inférieur à y
x<=y // x inférieur ou égale à y
```

Les opérateurs logiques :

Les opérateurs logiques sont utilisés pour comparer deux expressions/conditions et sont souvent utilisés dans les if. Ils renvoient TRUE (vrai) ou FALSE (faux) selon l'opération.

```
(x<5 && x>0) //vrai si x est inférieur à 5 ET supérieur à 0
(x>0 || y>0) // x est positif OU y est positif
!(x>0) // vrai si la condition est fausse (ici x négatif)
```

La plateforme Arduino

Les constantes :

Le langage Arduino possède des valeurs prédéfinies; elles sont appelées constantes. Elles sont utilisées pour rendre un programme plus simple à lire. Il en existe différents groupes.

True / False : Il s'agit de variable du type booléen. FALSE signifie que la constante est à 0 alors que TRUE désigne l'état différent de 0.

High / Low : Il s'agit des constantes utilisées pour lire ou pour écrire sur les broches, soit le niveau HIGH (ON, 1, 5 volts) soit le niveau LOW (OFF, 0, 0 volts).

digitalWrite(13,HIGH);

Input / Output : Ces constantes sont utilisées avec la fonction pinMode(), elles définissent le type de broche digitale.

pinMode(12, OUTPUT);

La plateforme Arduino

Les structures de contrôle :

if: La structure de test « if » (si) vérifie si une certaine condition est atteinte, par exemple si une valeur est égale à un certain nombre, et exécute des instructions contenues dans la structure. Si la condition est fausse, le programme quitte la structure et reprend à la ligne suivante. Le format du if est :

```
if(x==10) {  
    instructions;  
}
```

if ... else: if... else (si...sinon) est utilisé pour dire « sinon fait... ». Par exemple, si on veut tester une entrée digitale et faire une action si elle vaut HIGH et d'autres actions si elle vaut LOW, on écrira :

```
if(x==HIGH) {  
    instruction 1; instruction 2;  
    ...  
}  
else {  
    instruction a; instruction b;  
    ...  
}
```

La plateforme Arduino

Les structures de contrôle :

Switch ... case: La structure switch... case (commutateur de cas) permet de dresser une liste de tests ou de cas (comme le « if ») et d'exécuter le code correspondant si un des cas du test est vrai. La principale différence entre le switch... case et le if... else est que le switch... case permet de continuer les tests même si un des cas est vrai, contrairement au if...else qui quittera dans le même cas.

```
switch(Variable)  
{  
  case A: inst 1;  
  case B: inst 2;  
  case C: inst 3;  
  case D: inst 4;  
  default: instruction a;  
}
```

La plateforme Arduino

Les structures de contrôle :

For : La structure for (pour) est utilisée afin de répéter une série d'instructions (comprise entre les accolades) le nombre de fois voulu. Un compteur à incrémenter est souvent utilisé afin de sortir de la boucle une fois le nombre de tour fait. Cette structure est composée de trois parties séparées par un point-virgule (;) dans l'entête de la boucle :

```
for(int i=0;i<=10;i++)  
{  
  instructions;  
}
```

While : La boucle « while » (tant que) continuera infiniment tant que l'expression entre parenthèse est vraie. Quelque chose doit changer afin de modifier la condition sinon le programme ne sortira jamais de la boucle while.

```
while(x<20)  
{  
  instructions;  
  x++;  
}
```

La plateforme Arduino

Les structures de contrôle :

do ... while : La boucle do... while (faire... tant que) fonctionne de la même façon que la boucle while, à la seule exception que la condition est testée à la fin de la boucle, de ce fait le code dans la boucle s'exécute toujours au moins une fois

```
do
{
  instructions;
}
while(x>50);
```


La plateforme Arduino

Les entrées et sorties numériques :

pinMode(pin, mode) : Utilisée dans le void setup(), cette fonction permet de configurer une broche soit en entrée (INPUT) soit en sortie (OUTPUT).

pinMode(11, INPUT) ;// Le pin 11 sera utilisé comme entrée

digitalRead(pin) : Cette fonction permet de lire la valeur à la broche digitale indiquée. La valeur lue étant soit HIGH soit LOW. La broche ciblée ne peut être qu'une variable ou une constante comprise entre 0 et 13 (pour les Arduino Uno).

Val=digitalRead(11);//val prend la valeur lue sur le pin 11

digitalWrite(pin, value) : Il s'agit du contraire de digitalRead() : cette fonction permet de fixer une sortie digitale soit au niveau HIGH soit au niveau LOW. Cette sortie est comprise entre 0 et 13 sur Arduino Uno et doit être spécifiée soit par une variable soit par une constante.

digitalWrite(5, HIGH); // fixe la sortie du pin 5 à l'état HIGH (une tension de 5V)

La plateforme Arduino

Les entrées et sorties analogiques :

analogRead(pin) : Cette fonction permet de lire la valeur sur une sortie analogique avec 10 bits de résolution. Elle ne fonctionne que sur les broches analogiques (0-5) et renvoie un résultat compris entre 0 et 1023.

Val=analogRead(3); //val prend la valeur lue sur le pin 3

Note : Les broches analogiques ne sont pas des broches digitales, elle ne nécessite pas d'être déclarée comme des sorties ou des entrées.

analogWrite(pin, value) : analogWrite permet d'envoyer un pseudo-signal analogique en utilisant le matériel via les broches possédant un « pulse width modulation » (PWM ou modulation de largeur d'impulsions). Sur les Arduino Uno, cette fonctionnalité se trouve sur les broches 3, 5, 6, 9, 10 et 11. Ce signal est envoyé sous forme de valeur, comprise entre 0 et 255.

analogWrite(10, valeur); // envoie valeur sur le pin 10

delay(ms) : permet de marquer une pause dans le programme, le temps est en millisecondes

millis() : renvoie le nombre de millisecondes écoulé depuis que l'Arduino a commencé à exécuté le programme actuel.

La plateforme Arduino

La communication Série :

serial.begin(débit) : Permet d'ouvrir le port série et donne la vitesse de transmission (ou débit, en bits par seconde ou bauds). La vitesse usuelle est de 9600 bauds avec les ordinateurs mais d'autres vitesses sont supportées.

serial.begin(9600);

Note : Lorsque la liaison série est utilisée, les broches digitales 0 (RX) et 1 (TX) ne peuvent être utilisées en même temps.

serial.println(données): Permet d'envoyer des données sur le port série et d'automatiquement retourner à la ligne ensuite. La commande serial.print() fonctionne de la même façon mais sans le retour à la ligne.

serial.println(données):

La plateforme Arduino

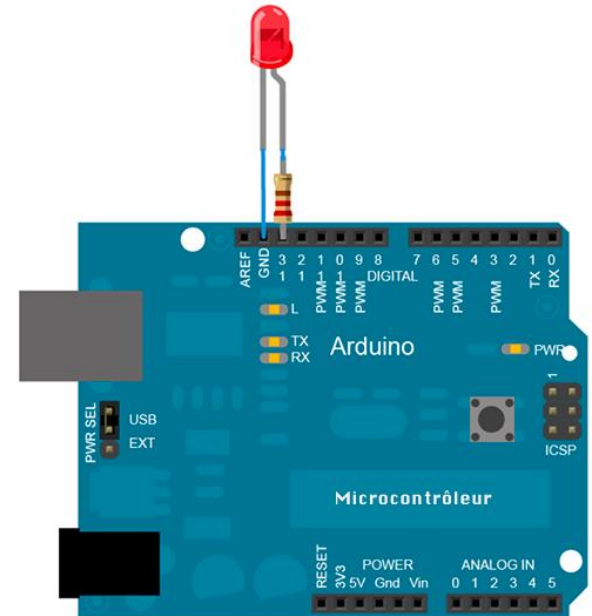
Exemple :

Nous allons choisir un exemple tout simple qui consiste à faire clignoter une LED (s'allume et s'éteint toutes les secondes).

```
void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);

  digitalWrite(13, LOW);
  delay(1000);
}
```



La plateforme Arduino

Exercices : (à faire !)

- 1) On désire réaliser un feu de signalisation routier. Pour info, le feu rouge dure 3 secondes et le feu orange dure 1 seconde .
- 2) On désire allumer une LED rouge à chaque fois qu'un bouton poussoir est appuyé et une LED verte sinon. Les deux LEDs ne sont jamais allumés en même temps.
- 3) On désire réaliser un système d'éclairage automatique avec une photorésistance. Si on a de l'obscurité (absence de lumière), une LED s'allume. Si on a de la lumière, la LED s'éteint.

